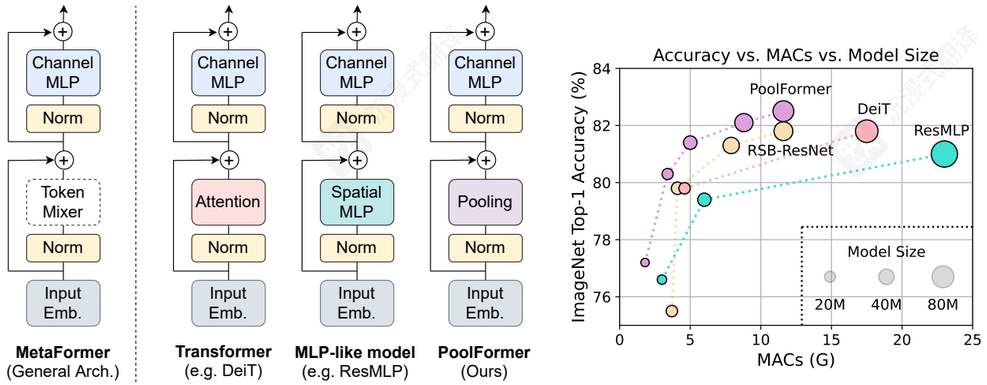
MetaFormer Is Actually What You Need for Vision

Weihao Yu[1,2*] Mi Luo[1] Pan Zhou[1] Chenyang Si[1] Yichen Zhou[1,2]
Xinchao Wang[2] Jiashi Feng[1] Shuicheng Yan[1]
[1]Sea AI Lab [2]National University of Singapore
weihaoyu6@gmail.com {luomi,zhoupan,sicy,zhouyc,fengjs,yansc}@sea.com xinchao@nus.edu.sg

Code: https://github.com/sail-sg/poolformer



(a) (b)

Figure 1. **MetaFormer and performance of MetaFormer-based models on ImageNet-1K validation set.** As shown in (a), we present *MetaFormer* as a general architecture abstracted from Transformers [56] by not specifying the token mixer. When using attention/spatial MLP as the token mixer, MetaFormer is instantiated as Transformer/MLP-like models. We argue that the competence of Transformer/MLP-like models primarily stems from the general architecture MetaFormer instead of the equipped specific token mixers. To demonstrate this, we exploit an embarrassingly simple non-parametric operator, *pooling*, to conduct extremely basic token mixing. Surprisingly, the resulted model *PoolFormer* consistently outperforms the well-tuned vision Transformer [17] baseline (DeiT [53]) and MLP-like [51] baseline (ResMLP [52]) as shown in (b), which well supports that MetaFormer is actually what we need to achieve competitive performance. RSB-ResNet in (b) means the results are from "ResNet Strikes Back" [59] where ResNet [24] are trained with improved training procedure for 300 epochs.

Abstract

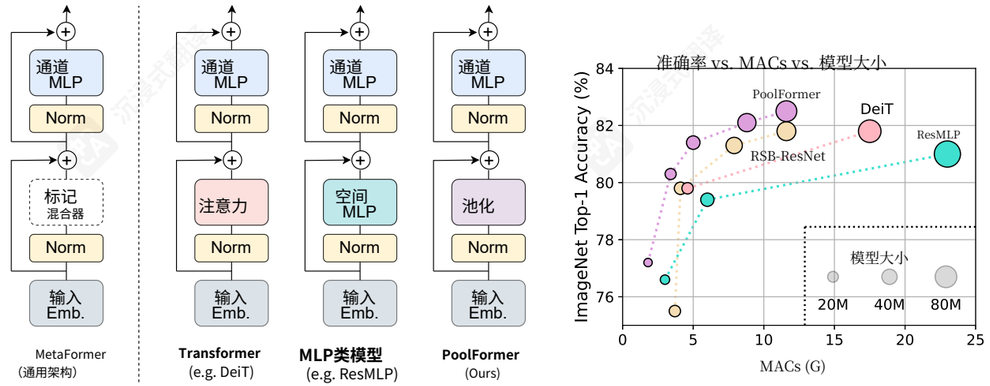
Transformers have shown great potential in computer vision tasks. A common belief is their attention-based token mixer module contributes most to their competence. However, recent works show the attention-based module in Transformers can be replaced by spatial MLPs and the resulted models still perform quite well. Based on this observation, we hypothesize that the general architecture of the Transformers, instead of the specific token mixer module, is more essential to the model's performance. To verify this, we deliberately replace the attention module in Transformers with an embarrassingly simple spatial pooling *operator to conduct only basic token mixing. Surprisingly, we observe that the derived model, termed as PoolFormer, achieves competitive performance on multiple computer vision tasks. For example, on ImageNet-1K, PoolFormer achieves 82.1% top-1 accuracy, surpassing well-tuned Vision Transformer/MLP-like baselines DeiT-B/ResMLP-B24 by 0.3%/1.1% accuracy with 35%/52% fewer parameters and 50%/62% fewer MACs. The effectiveness of PoolFormer verifies our hypothesis and urges us to initiate the concept of "MetaFormer", a general architecture abstracted from Transformers without specifying the token mixer. Based on the extensive experiments, we argue that MetaFormer is the key player in achieving superior results for recent Transformer and MLP-like models on vision tasks. This work calls for more future research dedicated to improving MetaFormer instead of focusing on the token mixer modules. Additionally, our proposed PoolFormer could serve as a starting baseline for future MetaFormer architecture design.*

*Work done during an internship at Sea AI Lab.

arXiv:2111.11418v3 [cs.CV] 4 Jul 2022

MetaFormer 实际上是您需要用于视觉的任务

Weihao Yu[1, 2*] Mi Luo[1] Pan Zhou[1] ChenyangSi[1] YichenZhou[1,2]Xinchao Wang[2] Jiashi Feng[1] ShuichengYan[II]Sea AI Lab [2]新加坡国立大学
weihaoyu6@gmail.com {罗米,周磐,徐驰,周宇辰,冯俊生,严胜}@sea.com 欣欣@nus.edu.sg
代码: https://github.com/sail-sg/poolformer



(a) (b)

图1. MetaFormer及其在ImageNet-1K验证集上的性能。如图(a)所示，我们将MetaFormer呈现为从Transformer中抽象出的通用架构，通过不指定token混合器。当使用注意力/空间MLP作为token混合器时，MetaFormer实例化为Transformer/MLP类模型。我们认为Transformer/MLP类模型的竞争力主要源于MetaFormer的通用架构，而非所配备的特定token混合器。为了证明这一点，我们利用一个极其简单的非参数算子——池化，进行极其基础的token混合。令人惊讶的是，所得模型PoolFormer始终优于经过优化的视觉Transformer基线（DeiT [53]）和MLP类基线（ResMLP [52]），如图(b)所示，这有力支持了MetaFormer实际上是我们实现竞争性性能所需的关键。RSB-ResNet（b）中的结果来自"ResNet反击"（v6），其中ResNet（v7）经过改进的训练流程训练了300个周期。

摘要

Transformer 在计算机视觉任务中展现出巨大潜力。一个普遍的观点是，它们基于注意力的 *token* 混合器模块对其能力贡献最大。然而，最近的研究表明，*Transformer* 中的基于注意力模块可以被空间 *MLP* 替换，而得到的模型仍然表现相当好。基于这一观察，我们假设 *Transformer* 的通用架构，而不是特定的 *token* 混合器模块，对模型性能更为关键。为了验证这一点，我们故意用一个非常简单的空间池化算子替换 *Transformer* 中的注意力模块，仅进行基本的 *token* 混合。令人惊讶的是，我们观察到得到的模型，称为 *PoolFormer*，在多个计算机视觉任务上实现了具有竞争力的性能。例如，在 *ImageNet-1K* 上，*PoolFormer* 达到 *82.1%* 的 *top-1* 准确率，比经过优化的 *Vi- sion Transformer/MLP* 类型的基线 *DeiT-B/ResMLP-B24* 高出 *0.3%/1.1%* 准确率，同时参数数量减少 *35%/52%*，*MACs* 减少 *50%/62%*。*PoolFormer* 的有效性验证了我们的假设，并促使我们提出了*"MetaFormer"*的概念，这是一种从 *Transformer* 中抽象出的通用架构，不指定 *token* 混合器。基于广泛的实验，我们认为 *MetaFormer* 是实现最近 *Transformer* 和 *MLP* 类模型在视觉任务上取得优异结果的关键因素。这项工作呼吁未来进行更多专门致力于着重改进*MetaFormer*，而不是专注于*token*混合器模块。此外，我们提出的*PoolFormer*可以作为未来*MetaFormer*架构设计的起点基线。

*在 Sea AI Lab 实习期间完成的工作。

1. Introduction

Transformers have gained much interest and success in the computer vision field [3, 8, 44, 55]. Since the seminal work of Vision Transformer (ViT) [17] that adapts pure Transformers to image classification tasks, many follow-up models are developed to make further improvements and achieve promising performance in various computer vision tasks [36, 53, 63].

The Transformer encoder, as shown in Figure 1(a), consists of two components. One is the attention module for mixing information among tokens and we term it as *token mixer*. The other component contains the remaining modules, such as channel MLPs and residual connections. By regarding the attention module as a specific token mixer, we further abstract the overall Transformer into a general architecture *MetaFormer* where the token mixer is not specified, as shown in Figure 1(a).

The success of Transformers has been long attributed to the attention-based token mixer [56]. Based on this common belief, many variants of the attention modules [13, 22, 57, 68] have been developed to improve the Vision Transformer. However, a very recent work [51] replaces the attention module completely with spatial MLPs as token mixers, and finds the derived MLP-like model can readily attain competitive performance on image classification benchmarks. The follow-up works [26, 35, 52] further improve MLP-like models by data-efficient training and specific MLP module design, gradually narrowing the performance gap to ViT and challenging the dominance of attention as token mixers.

Some recent approaches [32, 39, 40, 45] explore other types of token mixers within the MetaFormer architecture, and have demonstrated encouraging performance. For example, [32] replaces attention with Fourier Transform and still achieves around 97% of the accuracy of vanilla Transformers. Taking all these results together, it seems as long as a model adopts MetaFormer as the general architecture, promising results could be attained. We thus hypothesize *compared with specific token mixers, MetaFormer is more essential for the model to achieve competitive performance*.

To verify this hypothesis, we apply an extremely simple non-parametric operator, *pooling*, as the token mixer to conduct only basic token mixing. Astonishingly, this derived model, termed *PoolFormer*, achieves competitive performance, and even consistently outperforms well-tuned Transformer and MLP-like models, including DeiT [53] and ResMLP [52], as shown in Figure 1(b). More specifically, PoolFormer-M36 achieves 82.1% top-1 accuracy on ImageNet-1K classification benchmark, surpassing well-tuned vision Transformer/MLP-like baselines DeiT-B/ResMLP-B24 by 0.3%/1.1% accuracy with 35%/52% fewer parameters and 50%/62% fewer MACs. These results demonstrate that MetaFormer, even with a naive token mixer, can still deliver promising performance. We thus argue that MetaFormer is our *de facto* need for vision models which is more essential to achieve competitive performance rather than specific token mixers. Note that it does not mean the token mixer is insignificant. MetaFormer still has this abstracted component. It means token mixer is not limited to a specific type, e.g. attention.

The contributions of our paper are two-fold. Firstly, we abstract Transformers into a general architecture MetaFormer, and empirically demonstrate that the success of Transformer/MLP-like models is largely attributed to the MetaFormer architecture. Specifically, by only employing a simple non-parametric operator, pooling, as an extremely weak token mixer for MetaFormer, we build a simple model named PoolFormer and find it can still achieve highly competitive performance. We hope our findings inspire more future research dedicated to improving MetaFormer instead of focusing on the token mixer modules. Secondly, we evaluate the proposed PoolFormer on multiple vision tasks including image classification [14], object detection [34], instance segmentation [34], and semantic segmentation [67], and find it achieves competitive performance compared with the SOTA models using sophistic design of token mixers. The PoolFormer can readily serve as a good starting baseline for future MetaFormer architecture design.

2. Related work

Transformers are first proposed by [56] for translation tasks and then rapidly become popular in various NLP tasks. In language pre-training tasks, Transformers are trained on large-scale unlabeled text corpus and achieve amazing performance [2, 15]. Inspired by the success of Transformers in NLP, many researchers apply attention mechanism and Transformers to vision tasks [3, 8, 44, 55]. Notably, Chen *et al.* introduce iGPT [6] where the Transformer is trained to auto-regressively predict pixels on images for self-supervised learning. Dosovitskiy *et al.* propose Vision Transformer (ViT) with hard patch embedding as input [17]. They show that on supervised image classification tasks, a ViT pre-trained on a large propriety dataset (JFT dataset with 300 million images) can achieve excellent performance. DeiT [53] and T2T-ViT [63] further demonstrate that the ViT pre-trained on only ImageNet-1K (∼ 1.3 million images) from scratch can achieve promising performance. A lot of works have been focusing on improving the token mixing approach of Transformers by shifted windows [36], relative position encoding [61], refining attention map [68], or incorporating convolution [12, 21, 60], *etc*. In addition to attention-like token mixers, [51, 52] surprisingly find that merely adopting MLPs as token mixers can still achieve competitive performance. This discovery challenges the dominance of attention-based token mixers and triggers a heated discussion in the research community

1. 简介

Transformer在计算机视觉领域获得了广泛的兴趣和成功 [3, 8, 44, 55]。自视觉Transformer（ViT） [17]的最终工作以来，该工作将纯Transformer应用于图像分类任务，许多后续模型被开发出来以进一步改进并在各种计算机视觉任务 [36, 53, 63]中取得有前景的性能。

Transformer编码器，如图1(a)所示，由两个组件组成。一个是用于在标记之间混合信息的注意力模块，我们将其称为token混合器。另一个组件包含其余模块，例如通道MLPs和残差连接。通过将注意力模块视为一个特定的token混合器，我们将整体Transformer抽象为一般架构MetaFormer，其中token混合器未指定，如图1(a)所示。

Transformer的成功长期以来归因于基于注意力的token混合器 [56]。基于这种普遍信念，许多注意力模块的变体[13, 22, 57, 68] 已被开发出来以改进视觉Transformer。然而，最近的一篇工作 [51] 完全用空间MLPs替换了注意力模块作为token混合器，并发现由此得到的MLP类模型可以轻易在图像分类基准上达到有竞争力的性能。后续工作 [26, 35, 52] 通过数据高效训练和特定的MLP模块设计进一步改进MLP类模型，逐渐缩小了与ViT的性能差距，并挑战了注意力作为token混合器的统治地位。

一些最近的 [32, 39, 40, 45] 方法在MetaFormer架构内探索其他类型的token混合器，并展示了令人鼓舞的性能。例如， [32] 用傅里叶变换替换注意力机制，仍然达到了vanilla Transformer的约97%的准确率。综合所有这些结果，似乎只要模型采用MetaFormer作为通用架构，就有可能获得有前景的结果。因此，我们假设与特定的token混合器相比，MetaFormer对模型实现竞争性性能更为关键。

为验证这一假设，我们应用一个极其简单的非参数算子，池化，作为 token 混合器来执行仅基本的 token 混合。令人惊讶的是，这个衍生模型，称为 PoolFormer，实现了具有竞争力的性能，甚至始终优于经过优化的 Transformer 和 MLP 类模型，包括 DeiT[53] 和 ResMLP [52]，，如图 1(b) 所示。更具体地说，PoolFormer-M36 在 ImageNet-1K 分类基准上实现了 82.1% 的 top-1 准确率，在参数数量上比经过优化的视觉 Transformer/MLP 类基线 DeiT-B/ResMLP-B24 少 35%/52%，在 MACs 上少 50%/62%，同时准确率分别提高了 0.3%/1.1%。这些结果表明，MetaFormer，即使使用一个简单的 token 混合器，仍然可以提供有前景的性能。因此，我们主张 MetaFormer 是我们实际需要的视觉模型，它比特定的 token 混合器更关键于实现具有竞争力的性能。请注意，这并不意味着 token 混合器不重要。MetaFormer 仍然具有这个抽象的组件。这意味着 token 混合器不限于特定类型，例如注意力。

本文的贡献有两方面。首先，我们将Transformer抽象为通用架构MetaFormer，并实证证明Transformer/MLP类模型的成功很大程度上归因于MetaFormer架构。具体来说，我们仅采用简单的非参数算子——池化，作为MetaFormer的极弱token混合器，构建了一个名为PoolFormer的简单模型，发现它仍然可以实现高度竞争性的性能。我们希望我们的发现能激励更多未来研究致力于改进MetaFormer，而不是专注于token混合器模块。其次，我们在包括图像分类 [14], 目标检测 [34], 实例分割 [34], 和语义分割 [67],的多个视觉任务上评估了提出的PoolFormer，发现它在使用精心设计的token混合器的SOTA模型中实现了竞争性性能。PoolFormer可以作为一个良好的起点基线，用于未来MetaFormer架构设计。

2. 相关工作

Transformer是由 [56] 首次提出的，用于翻译任务，然后迅速在各种NLP任务中流行起来。在语言预训练任务中，Transformer在大型无标签文本语料库上进行训练，并取得了惊人的性能 [2, 15]。受Transformer在NLP中成功的启发，许多研究人员将注意力机制和Transformer应用于视觉任务 [3, 8, 44, 55]。值得注意的是，Chen等人引入了iGPT [6]，其中Transformer被训练为自回归地预测图像上的像素，用于自监督学习。Dosovitskiy等人提出了具有硬patch嵌入作为输入的视觉Transformer（ViT） [17]。他们表明，在监督图像分类任务上，一个在大型专有数据集（包含3亿张图像的JFT数据集）上预训练的ViT可以实现优异的性能。DeiT [53]和T2T-ViT [63]进一步证明了，仅从零开始预训练在ImageNet-1K（∼ 1.3亿张图像）上的ViT可以实现有前景的性能。许多工作都集中在改进Transformer的token混合方法，通过移位窗口 [36], 相对位置编码 [61], 细化注意力图 [68],或结合卷积 [12, 21, 60],等。除了类似注意力的token混合器， [51, 52] 意外地发现，仅采用MLPs作为token混合器仍然可以实现有竞争力的性能。这一发现挑战了基于注意力的token混合器的统治地位，并在研究界引发了热烈的讨论

Algorithm 1 Pooling for PoolFormer, PyTorch-like Code

```
import torch.nn as nn

class Pooling(nn.Module):
    def __init__(self, pool_size=3):
        super().__init__()
        self.pool = nn.AvgPool2d(
            pool_size, stride=1,
            padding=pool_size//2,
            count_include_pad=False,
        )
    def forward(self, x):
        """
        [B, C, H, W] = x.shape
        Subtraction of the input itself is added
        since the block already has a
        residual connection.
        """
        return self.pool(x) - x
```

about which token mixer is better [7, 26]. However, the target of this work is neither to be engaged in this debate nor to design new complicated token mixers to achieve new state of the art. Instead, we examine a fundamental question: What is truly responsible for the success of the Transformers and their variants? Our answer is the general architecture *i.e.*, MetaFormer. We simply utilize pooling as basic token mixers to probe the power of MetaFormer.

Contemporarily, some works contribute to answering the same question. Dong *et al.* prove that without residual connections or MLPs, the output converges doubly exponentially to a rank one matrix [16]. Raghu *et al.* [43] compare the feature difference between ViT and CNNs, finding that self-attention allows early gathering of global information while residual connections greatly propagate features from lower layers to higher ones. Park *et al.* [42] shows that multi-head self-attentions improve accuracy and generalization by flattening the loss landscapes. Unfortunately, they do not abstract Transformers into a general architecture and study them from the aspect of general framework.

3. Method

3.1. MetaFormer

We present the core concept "MetaFormer" for this work at first. As shown in Figure 1, abstracted from Transformers [56], MetaFormer is a general architecture where the token mixer is not specified while the other components are kept the same as Transformers. The input I is first processed by input embedding, such as patch embedding for ViTs [17],

$$X = \mathrm{InputEmb}(I), \tag{1}$$

where $X \in \mathbb{R}^{N \times C}$ denotes the embedding tokens with sequence length N and embedding dimension C.

Then, embedding tokens are fed to repeated MetaFormer blocks, each of which includes two residual sub-blocks. Specifically, the first sub-block mainly contains a token mixer to communicate information among tokens and this sub-block can be expressed as

$$Y = \mathrm{TokenMixer}(\mathrm{Norm}(X)) + X, \tag{2}$$

where $\mathrm{Norm}(\cdot)$ denotes the normalization such as Layer Normalization [1] or Batch Normalization [28]; $\mathrm{TokenMixer}(\cdot)$ means a module mainly working for mixing token information. It is implemented by various attention mechanism in recent vision Transformer models [17, 63, 68] or spatial MLP in MLP-like models [51, 52]. Note that the main function of the token mixer is to propagate token information although some token mixers can also mix channels, like attention.

The second sub-block primarily consists of a two-layered MLP with non-linear activation,

$$Z = \sigma(\mathrm{Norm}(Y)W_1)W_2 + Y, \tag{3}$$

where $W_1 \in \mathbb{R}^{C \times rC}$ and $W_2 \in \mathbb{R}^{rC \times C}$ are learnable parameters with MLP expansion ratio r; $\sigma(\cdot)$ is a non-linear activation function, such as GELU [25] or ReLU [41].

Instantiations of MetaFormer. MetaFormer describes a general architecture with which different models can be obtained immediately by specifying the concrete design of the token mixers. As shown in Figure 1(a), if the token mixer is specified as attention or spatial MLP, MetaFormer then becomes a Transformer or MLP-like model respectively.

3.2. PoolFormer

From the introduction of Transformers [56], lots of works attach much importance to the attention and focus on designing various attention-based token mixer components. In contrast, these works pay little attention to the general architecture, *i.e.*, the MetaFormer.

In this work, we argue that this MetaFormer general architecture contributes mostly to the success of the recent Transformer and MLP-like models. To demonstrate it, we deliberately employ an embarrassingly simple operator, pooling, as the token mixer. This operator has no learnable parameters and it just makes each token averagely aggregate its nearby token features.

Since this work is targeted at vision tasks, we assume the input is in channel-first data format, *i.e.*, $T \in \mathbb{R}^{C \times H \times W}$. The pooling operator can be expressed as

$$T'_{:,i,j} = \frac{1}{K \times K} \sum_{p,q=1}^{K} T_{:,i+p-\frac{K+1}{2},i+q-\frac{K+1}{2}} - T_{:,i,j}, \tag{4}$$

where K is the pooling size. Since the MetaFormer block already has a residual connection, subtraction of the input itself is added in Equation (4). The PyTorch-like code of the pooling is shown in Algorithm 1.

Algorithm 1 Pooling for PoolFormer, PyTorch-like Code

```
import torch.nn as nn

class Pooling(nn.Module):
    def __init__(self, pool_size=3):
        super().__init__()
        self.pool = nn.AvgPool2d(
            pool_size, stride=1,
            padding=pool_size//2,
            count_include_pad=False,
        )
    def forward(self, x):
        """
        [B, C, H, W] = x.shape
        Subtraction of the input itself is added
        since the block already has a
        residual connection.
        """
        return self.pool(x) - x
```


关于哪个token混合器更好 [7,26]。然而，这项工作的目标既不是参与这场辩论，也不是设计新的复杂token混合器以实现新的SOTA。相反，我们考察了一个基本问题：Transformer及其变体的成功真正是由什么负责？我们的答案是通用架构，即MetaFormer。我们简单地利用池化作为基本的token混合器来探测MetaFormer的力量。

目前，一些工作致力于回答同一个问题。Dong 等人证明，没有残差连接或 MLP，输出会以双指数速度收敛到一个秩一矩阵 [16]。Raghu 等人 [43] 比较了 ViT 和 CNN 的特征差异，发现自注意力允许早期收集全局信息，而残差连接极大地将特征从低层传播到高层。Park 等人 [42] 表明，多头自注意力通过平滑损失函数曲面来提高准确性和泛化能力。不幸的是，他们没有将 Transformer 抽象为通用架构，也没有从通用框架的角度来研究它们。

3. 方法

3.1. MetaFormer

我们首先介绍了这项工作的核心概念“MetaFormer”。如图 1 所示，从 Transformer[56]，抽象而来，MetaFormer 是一个通用架构，其中 token 混合器没有指定，而其他组件与 Transformer 相同。输入 I 首先由输入嵌入处理，例如 ViT 的 patch 嵌入 [17]，

$$X = \mathrm{InputEmb}(I), \tag{1}$$

where $X \in \mathbb{R}^{N \times C}$ 表示具有序列长度 N 和嵌入维度 C 的嵌入token。

然后，嵌入标记被输入到重复的MetaFormer块中，每个块包含两个残差子模块。具体来说，第一个子模块主要包含一个标记token混合器来在标记之间传递信息，并且这个子模块可以表示为

$$Y = \mathrm{TokenMixer}(\mathrm{Norm}(X)) + X, \tag{2}$$

where Norm(·) 表示归一化，例如层归一化 [1] 或批量归一化 [28]；TokenMixer(·) 表示主要用于混合token信息的模块。它在最近的视觉Transformer模型 [17,63,68] 或MLP类模型 [51,52] 中的空间MLP中实现。请注意，token混合器的主要功能是传播token信息，尽管一些token混合器也可以混合通道，例如注意力。

第二个子块主要包含一个具有非线性激活的两层 MLP，

$$Z = \sigma(\mathrm{Norm}(Y)W_1)W_2 + Y, \tag{3}$$

where $W_1 \in \mathbb{R}^{C \times rC}$ 和 $W_2 \in \mathbb{R}^{rC \times C}$ 是具有MLP扩展比例 r 的可学习参数；$\sigma(\cdot)$ 是一个非线性激活函数，例如 GELU [25] 或ReLU [41]。

MetaFormer的实例化。MetaFormer描述了一种通用架构，通过指定token混合器的具体设计，可以立即获得不同的模型。如图1(a)所示，如果token混合器指定为注意力或空间MLP，MetaFormer分别成为Transformer或MLP类模型。

3.2. PoolFormer

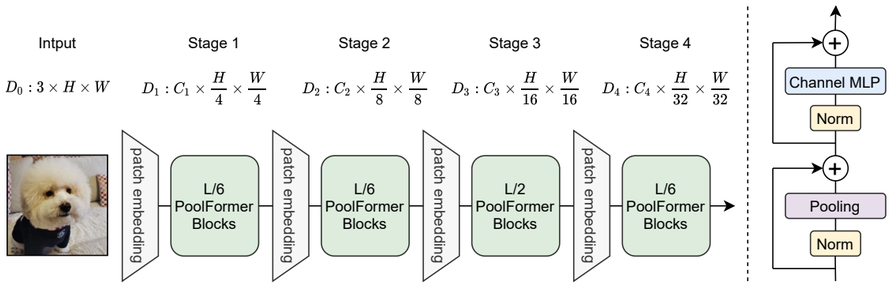
从Transformers的介绍 [56]，来看，大量工作非常重视注意力，并专注于设计各种基于注意力的token混合器组件。相比之下，这些工作很少关注通用架构，即MetaFormer。

在这项工作中，我们认为这种MetaFormer通用架构主要促成了最近Transformer和MLP类模型的成功。为了证明这一点，我们故意采用了一个非常简单的算子——池化，作为token混合器。这个算子没有可学习的参数，它只是让每个token平均地聚合其附近的token特征。

由于这项工作针对的是视觉任务，我们假设输入是通道优先数据格式，即 $T \in \mathbb{R}^{C \times H \times W}$。池化算子可以表示为

$$T'_{:,i,j} = \frac{1}{K \times K} \sum_{p,q=1}^{K} T_{:,i+p-\frac{K+1}{2},i+q-\frac{K+1}{2}} - T_{:,i,j}, \tag{4}$$

其中 K 是池化大小。由于 MetaFormer 模块已经具有残差连接，因此输入本身的减法在公式 (4) 中被添加。池化的类似 PyTorch 的代码显示在算法 1 中。



(a) Overal framework with L PoolFormer blocks (b) PoolFormer block

Figure 2. **(a) The overall framework of PoolFormer.** Similar to [24, 36, 57], PoolFormer adopts hierarchical architecture with 4 stages. For a model with L PoolFormer blocks, stage [1, 2, 3, 4] have [L/6, L/6, L/2, L/6] blocks, respectively. The feature dimension D_i of stage i is shown in the figure. **(b) The architecture of PoolFormer block.** Compared with Transformer block, it replaces attention with extremely simple non-parametric operator, pooling, to conduct only basic token mixing.

As well known, self-attention and spatial MLP have computational complexity quadratic to the number of tokens to mix. Even worse, spatial MLPs bring much more parameters when handling longer sequences. As a result, self-attention and spatial MLPs usually can only process hundreds of tokens. In contrast, the pooling needs a computational complexity linear to the sequence length without any learnable parameters. Thus, we take advantage of pooling by adopting a hierarchical structure similar to traditional CNNs [24,31,49] and recent hierarchical Transformer variants [36,57]. Figure 2 shows the overall framework of PoolFormer. Specifically, PoolFormer has 4 stages with $\frac{H}{4} \times \frac{W}{4}$, $\frac{H}{8} \times \frac{W}{8}$, $\frac{H}{16} \times \frac{W}{16}$, and $\frac{H}{32} \times \frac{W}{32}$ tokens respectively, where H and W represent the width and height of the input image. There are two groups of embedding size: 1) small-sized models with embedding dimensions of 64, 128, 320, and 512 responding to the four stages; 2) medium-sized models with embedding dimensions 96, 192, 384, and 768. Assuming there are L PoolFormer blocks in total, stages 1, 2, 3, and 4 will contain $L/6$, $L/6$, $L/2$, and $L/6$ PoolFormer blocks respectively. The MLP expansion ratio is set as 4. According to the above simple model scaling rule, we obtain 5 different model sizes of PoolFormer and their hyper-parameters are shown in Table 1.

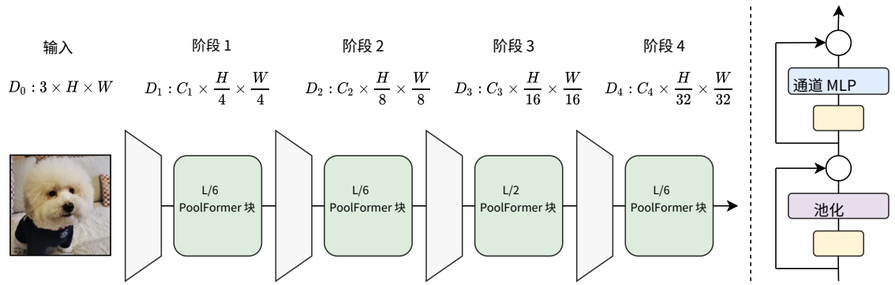
Stage	#Tokens	Layer Specification		PoolFormer				
				S12	S24	S36	M36	M48
1	$\frac{H}{4} \times \frac{W}{4}$	Patch Embedding	Patch Size	7×7, stride 4				
			Embed. Dim.	64			96	
		PoolFormer Block	Pooling Size	3×3, stride 1				
			MLP Ratio	4				
			# Block	2	4	6	6	8
2	$\frac{H}{8} \times \frac{W}{8}$	Patch Embedding	Patch Size	3×3, stride 2				
			Embed. Dim.	128			192	
		PoolFormer Block	Pooling Size	3×3, stride 1				
			MLP Ratio	4				
			# Block	2	4	6	6	8
3	$\frac{H}{16} \times \frac{W}{16}$	Patch Embedding	Patch Size	3×3, stride 2				
			Embed. Dim.	320			384	
		PoolFormer Block	Pooling Size	3×3, stride 1				
			MLP Ratio	4				
			# Block	6	12	18	18	24
4	$\frac{H}{32} \times \frac{W}{32}$	Patch Embedding	Patch Size	3×3, stride 2				
			Embed. Dim.	512			768	
		PoolFormer Block	Pooling Size	3×3, stride 1				
			MLP Ratio	4				
			# Block	2	4	6	6	8
Parameters (M)				11.9	21.4	30.8	56.1	73.4
MACs (G)				1.8	3.4	5.0	8.8	11.6

Table 1. **Configurations of different PoolFormer models.** There are two groups of embedding dimensions, *i.e.*, small size with [64, 128, 320, 512] dimensions and medium size with [96, 196, 384, 768]. Notation "S24" means the model is in small size of embedding dimensions with 24 PoolFormer blocks in total. The numbers of MACs are counted by `fvcore` [19] library.

4. Experiments

4.1. Image classification

Setup. ImageNet-1K [14] is one of the most widely used datasets in computer vision. It contains about 1.3M training images and 50K validation images, covering common 1K classes. Our training scheme mainly follows [53] and [54]. Specifically, MixUp [65], CutMix [64], CutOut [66] and RandAugment [11] are used for data augmentation. The models are trained for 300 epochs using AdamW optimizer [29,37] with weight decay 0.05 and peak learning rate $\text{lr} = 1e^{-3} \cdot \text{batch size}/1024$ (batch size 4096 and learning rate $4e^{-3}$ are used in this paper). The number of warmup epochs is 5 and cosine schedule is used to decay the learning rate. Label Smoothing [50] is set as 0.1. Dropout is disabled but stochastic depth [27] and LayerScale [54] are



(a) 整体框架与L个PoolFormer模块 (b) PoolFormer模块

图2。(a) PoolFormer的整体框架。类似于 [24,36,57], PoolFormer采用分层架构，包含4个阶段。对于包含L个PoolFormer模块的模型，阶段 [1, 2, 3, 4] 分别包含 [L/6、L/6、L/2、L/6] 个模块。阶段 i 的特征维度 D_i 如图所示。(b) PoolFormer模块的架构。与Transformer模块相比，它用非常简单的非参数算子——池化，替代注意力机制，仅进行基本的标记混合。

众所周知，自注意力和空间 MLP 的计算复杂度与混合标记的数量呈平方关系。更糟糕的是，当处理更长的序列时，空间 MLP 会引入更多参数。因此，自注意力和空间 MLP 通常只能处理数百个标记。相比之下，池化的计算复杂度与序列长度呈线性关系，且没有任何可学习参数。因此，我们通过采用类似于传统 CNNs [24,31,49] 和近期分层 Transformer 变体 [36,57] 的分层结构来利用池化。图 2 显示了 PoolFormer 的整体框架。具体来说，PoolFormer 有 4 个阶段，分别为 $\frac{H}{4} \times \frac{W}{4}$、$\frac{H}{8} \times \frac{W}{8}$、$\frac{H}{16} \times \frac{W}{16}$ 和 $\frac{H}{32} \times \frac{W}{32}$ 个标记，其中 H 和 W 表示输入图像的宽度和高度。嵌入大小分为两组：1) 小型模型，嵌入维度为 64、128、320 和 512，对应于四个阶段；2) 中型模型，嵌入维度为 96、192、384 和 768。假设总共有 L 个 PoolFormer 模块，阶段 1、2、3 和 4 将分别包含 $L/6$、$L/6$、$L/2$ 和 $L/6$ 个 PoolFormer 模块。MLP 扩展率设置为 4。根据上述简单的模型缩放规则，我们获得了 5 种不同的 PoolFormer 模型大小，它们的超参数显示在表 1 中。

阶段	#标记	层规范		PoolFormer				
				S12	S24	S36	M36	M48
1	$\frac{H}{4} \times \frac{W}{4}$	补丁嵌入	补丁大小	7×7, 步长 4				
			嵌入维度	64			96	
		PoolFormer 块	池化大小	3×3, 步长1				
			MLP比率	4				
			#块	2	4	6	6	8
2	$\frac{H}{8} \times \frac{W}{8}$	块嵌入	补丁大小	3×3, 步长 2				
			嵌入维度	128			192	
		PoolFormer 块	池化大小	3×3, 步长1				
			MLP比率	4				
			#块	2	4	6	6	8
3	$\frac{H}{16} \times \frac{W}{16}$	块嵌入	补丁大小	3×3, 步长 2				
			嵌入维度	320			384	
		PoolFormer 块	池化大小	3×3, 步长1				
			MLP比率	4				
			#块	6	12	18	18	24
4	$\frac{H}{32} \times \frac{W}{32}$	块嵌入	补丁大小	3×3, 步长2				
			嵌入维度	512			768	
		PoolFormer 块	池化大小	3×3, 步长1				
			MLP比率	4				
			#块	2	4	6	6	8
参数（M）				11.9	21.4	30.8	56.1	73.4
MACs（G）				1.8	3.4	5.0	8.8	11.6

表1. 不同PoolFormer模型的配置。嵌入维度分为两组，即小尺寸的 [64,128, 320, 512]维度和中等尺寸的 [96, 196, 384, 768]维度。符号“S24”表示模型嵌入维度为小尺寸，共有24个PoolFormer模块。MACs的数量由fvcore [19] 库计算得出。

4. 实验

4.1. 图像分类

设置。ImageNet-1K [14] 是计算机视觉中广泛使用的数据集之一。它包含约130万训练图像和5万个验证图像，涵盖常见的1K个类别。我们的训练方案主要遵循 [53] 和 [54]。具体来说，MixUp [65], CutMix [64], CutOut [66]和RandAugment [11]用于数据增强。模型使用AdamW优化器训练300个周期。

mizer [29,37] with 权重衰减 0.05 和 峰值学习率 $\text{lr} = 1e^{-3} \cdot$批大小$/1024$ (批大小 4096 和 学习率 $4e^{-3}$ 在本文中使用)。预热周期为 5，使用余弦调度来衰减学习率。标签平滑 [50] 设置为 0.1。Dropout 被禁用，但随机深度 [27] 和 LayerScale [54] 是

General Arch.	Token Mixer	Outcome Model	Image Size	Params (M)	MACs (G)	Top-1 (%)
Convolutional Neural Netowrks	—	RSB-ResNet-18 [24, 59]	224	12	1.8	70.6
		RSB-ResNet-34 [24, 59]	224	22	3.7	75.5
		RSB-ResNet-50 [24, 59]	224	26	4.1	79.8
		RSB-ResNet-101 [24, 59]	224	45	7.9	81.3
		RSB-ResNet-152 [24, 59]	224	60	11.6	81.8
MetaFormer	Attention	ViT-B/16* [17]	224	86	17.6	79.7
		ViT-L/16* [17]	224	307	63.6	76.1
		DeiT-S [53]	224	22	4.6	79.8
		DeiT-B [53]	224	86	17.5	81.8
		PVT-Tiny [57]	224	13	1.9	75.1
		PVT-Small [57]	224	25	3.8	79.8
		PVT-Medium [57]	224	44	6.7	81.2
		PVT-Large [57]	224	61	9.8	81.7
	Spatial MLP	MLP-Mixer-B/16 [51]	224	59	12.7	76.4
		ResMLP-S12 [52]	224	15	3.0	76.6
		ResMLP-S24 [52]	224	30	6.0	79.4
		ResMLP-B24 [52]	224	116	23.0	81.0
		Swin-Mixer-T/D24 [36]	256	20	4.0	79.4
		Swin-Mixer-T/D6 [36]	256	23	4.0	79.7
		Swin-Mixer-B/D24 [36]	224	61	10.4	81.3
		gMLP-S [35]	224	20	4.5	79.6
		gMLP-B [35]	224	73	15.8	81.6
	Pooling	PoolFormer-S12	224	12	1.8	77.2
		PoolFormer-S24	224	21	3.4	80.3
		PoolFormer-S36	224	31	5.0	81.4
		PoolFormer-M36	224	56	8.8	82.1
		PoolFormer-M48	224	73	11.6	82.5

Table 2. **Performance of different types of models on ImageNet-1K classification.** All these models are only trained on the ImageNet-1K training set and the accuracy on the validation set is reported. RSB-ResNet means the results are from "ResNet Strikes Back" [59] where ResNet [24] is trained with improved training procedure for 300 epochs. * denotes results of ViT trained with extra regularization from [51]. The numbers of MACs of PoolFormer are counted by `fvcore` [19] library.

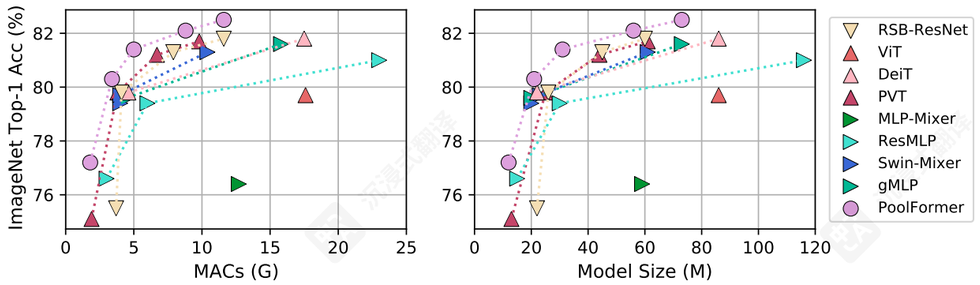


Figure 3. **ImageNet-1K validation accuracy *vs*. MACs/Model Size.** RSB-ResNet means the results are from "ResNet Strikes Back" [59] where ResNet [24] is trained with improved training procedure for 300 epochs.

used to help train deep models. We modified Layer Normalization [1] to compute the mean and variance along token and channel dimensions compared to only channel dimension in vanilla Layer Normalization. Modified Layer Normalization (MLN) can be implemented for channel-first data format with GroupNorm API in PyTorch by specifying the group number as 1. MLN is preferred by PoolFormer as shown in Section 4.4. See the appendix for more details on hyper-parameters. Our implementation is based on the `Timm` codebase [58] and the experiments are run on TPUs.

Results. Table 2 shows the performance of PoolFormers on ImageNet classification. Qualitative results are shown in the appendix. Surprisingly, despite the simple pooling token mixer, PoolFormers can still achieve highly competitive performance compared with CNNs and other MetaFormer-

通用架构	token混合器	结果模型	图像大小	参数（M）	MACs（G）	Top-1（%）
卷积 神经网络	—	RSB-ResNet-18 [24, 59]	224	12	1.8	70.6
		RSB-ResNet-34 [24, 59]	224	22	3.7	75.5
		RSB-ResNet-50 [24, 59]	224	26	4.1	79.8
		RSB-ResNet-101 [24, 59]	224	45	7.9	81.3
		RSB-ResNet-152 [24, 59]	224	60	11.6	81.8
MetaFormer	注意力	ViT-B/16* [17]	224	86	17.6	79.7
		ViT-L/16* [17]	224	307	63.6	76.1
		DeiT-S [53]	224	22	4.6	79.8
		DeiT-B [53]	224	86	17.5	81.8
		PVT-Tiny [57]	224	13	1.9	75.1
		PVT-Small [57]	224	25	3.8	79.8
		PVT-Medium [57]	224	44	6.7	81.2
		PVT-Large [57]	224	61	9.8	81.7
	空间MLP	MLP-Mixer-B/16 [51]	224	59	12.7	76.4
		ResMLP-S12 [52]	224	15	3.0	76.6
		ResMLP-S24 [52]	224	30	6.0	79.4
		ResMLP-B24 [52]	224	116	23.0	81.0
		Swin-Mixer-T/D24 [36]	256	20	4.0	79.4
		Swin-Mixer-T/D6 [36]	256	23	4.0	79.7
		Swin-Mixer-B/D24 [36]	224	61	10.4	81.3
		gMLP-S [35]	224	20	4.5	79.6
		gMLP-B [35]	224	73	15.8	81.6
	池化	PoolFormer-S12	224	12	1.8	77.2
		PoolFormer-S24	224	21	3.4	80.3
		PoolFormer-S36	224	31	5.0	81.4
		PoolFormer-M36	224	56	8.8	82.1
		PoolFormer-M48	224	73	11.6	82.5

表 2. ImageNet-1K 分类中不同类型模型的表现。所有这些模型仅在 ImageNet-1K 训练集上训练，并报告了验证集上的准确率。RSB-ResNet 表示结果来自“ResNet反击” [59]，其中 ResNet [24] 使用改进的训练程序训练 300 个周期。 * 表示 ViT 使用额外正则化的结果，来自 [51]。PoolFormer 的 MACs 数量由 fvcore [19] 库计算。

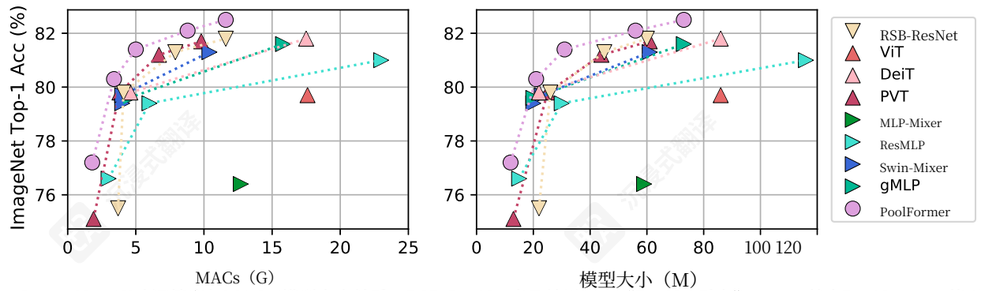


图3。ImageNet-1K验证精度与MACs/模型大小关系。RSB-ResNet表示结果来自“ResNet反击” [59]，其中ResNet [24] 使用改进的训练流程训练了300个周期。

用于帮助训练深度模型。我们修改了Layer Normalization [1]，使其沿token和通道维度计算均值和方差，而vanilla Layer Normalization仅在通道维度上计算。改进层归一化（MLN）可以通过PyTorch中的GroupNorm API以通道优先数据格式实现，通过指定组数为1。PoolFormer如图4.4所示更倾向于使用MLN。更多细节请参见附录

关于超参数。我们的实现基于Timm codebase [58]，实验在TPUs上运行。

结果。表2显示了PoolFormers在ImageNet分类上的性能。定性结果在附录中展示。令人惊讶的是，尽管是简单的池化token混合器，PoolFormers仍然可以实现与CNNs和其他MetaFormer-

like models. For example, PoolFormer-S24 reaches the top-1 accuracy of more than 80 while only requiring 21M parameters and 3.4G MACs. Comparatively, the well-established ViT baseline DeiT-S [53], attains slightly worse accuracy of 79.8 and requires 35% more MACs (4.6G). To obtain similar accuracy, MLP-like model ResMLP-S24 [52] needs 43% more parameters (30M) as well as 76% more computation (6.0G) while only 79.4 accuracy is attained. Even compared with more improved ViT and MLP-like variants [35, 57], PoolFormer still shows better performance. Specifically, the pyramid Transformer PVT-Medium obtains 81.2 top-1 accuracy with 44M parameters and 6.7G MACs while PoolFormer-S36 reaches 81.4 with 30% fewer parameters (31M) and 25% fewer MACs (5.0G) than those of PVT-Medium.

Besides, compared with RSB-ResNet ("ResNet Strikes Back") [59] where ResNet [24] is trained with improved training procedure for the same 300 epochs, PoolFormer still performs better. With $\sim$ 22M parameters/3.7G MACs, RSB-ResNet-34 [59] gets 75.5 accuracy while PoolFormer-S24 can obtain 80.3. Since the local spatial modeling ability of the pooling layer is much worse than the neural convolution layer, the competitive performance of PoolFormer can only be attributed to its general architecture MetaFormer.

With the pooling operator, each token evenly aggregates the features from its nearby tokens. Thus it is an extremely basic token mixing operation. However, the experiment results show that even with this embarrassingly simple token mixer, MetaFormer still obtains highly competitive performance. Figure 3 clearly shows that PoolFormer surpasses other models with fewer MACs and parameters. This finding conveys that the general architecture MetaFormer is actually what we need when designing vision models. By adopting MetaFormer, it is guaranteed that the derived models would have the potential to achieve reasonable performance.

4.2. Object detection and instance segmentation

Setup. We evaluate PoolFormer on the challenging COCO benchmark [34] that includes 118K training images (`train2017`) and 5K validation images (`val2017`). The models are trained on training set and the performance on validation set is reported. PoolFormer is employed as the backbone for two standard detectors, *i.e.*, RetinaNet [33] and Mask R-CNN [23]. ImageNet pre-trained weights are utilized to initialize the backbones and Xavier [20] to initialize the added layers. AdamW [29,37] is adopted for training with an initial learning rate of 1×10^{-4} and batch size of 16. Following [23, 33], we employ $1\times$ training schedule, *i.e.*, training the detection models for 12 epochs. The training images are resized into shorter side of 800 pixels and longer side of no more than 1,333 pixels. For testing, the shorter side of the images is also resized to 800 pixels. The implementation is based on the `mmdetection` [4] codebase and the experiments are run on 8 NVIDIA A100 GPUs.

Results. Equipped with RetinaNet for object detection, PoolFormer-based models consistently outperform their comparable ResNet counterparts as shown in Table 3. For instance, PoolFormer-S12 achieves 36.2 AP, largely surpassing that of ResNet-18 (31.8 AP). Similar results are observed for those models based on Mask R-CNN on object detection and instance segmentation. For example, PoolFormer-S12 largely surpasses ResNet-18 (bounding box AP 37.3 *vs*. 34.0, and mask AP 34.6 *vs*. 31.2). Overall, for COCO object detection and instance segmentation, PoolForemrs achieve competitive performance, consistently outperforming those counterparts of ResNet.

4.3. Semantic segmentation

Setup. ADE20K [67], a challenging scene parsing benchmark, is selected to evaluate the models for semantic segmentation. The dataset includes 20K and 2K images in the training and validation set, respectively, covering 150 fine-grained semantic categories. PoolFormers are evaluated as backbones equipped with Semantic FPN [30]. ImageNet-1K trained checkpoints are used to initialize the backbones while Xavier [20] is utilized to initialize other newly added layers. Common practices [5, 30] train models for 80K iterations with a batch size of 16. To speed up training, we double the batch size to 32 and decrease the iteration number to 40K. The AdamW [29,37] is employed with an initial learning rate of 2×10^{-4} that will decay in the polynomial decay schedule with a power of 0.9. Images are resized and cropped into 512×512 for training and are resized to shorter side of 512 pixels for testing. Our implementation is based on the `mmsegmentation` [10] codebase and the experiments are conducted on 8 NVIDIA A100 GPUs.

Results. Table 4 shows the ADE20K semantic segmentation performance of different backbones using FPN [30]. PoolFormer-based models consistently outperform the models with backbones of CNN-based ResNet [24] and ResNeXt [62] as well as Transformer-based PVT. For instance, PoolFormer-12 achieves mIoU of 37.1, 4.3 and 1.5 better than ResNet-18 and PVT-Tiny, respectively.

These results demonstrate that our PoorFormer which serves as backbone can attain competitive performance on semantic segmentation although it only utilizes pooling for basically communicating information among tokens. This further indicates the great potential of MetaFormer and supports our claim that MetaFormer is actually what we need.

4.4. Ablation studies

The experiments of ablation studies are conducted on ImageNet-1K [14]. Table 5 reports the ablation study of PoolFormer. We discuss the ablation below according to the following aspects.

类模型相比具有高度竞争力的性能。例如，PoolFormer-S24达到了超过80%的top-1准确率，同时仅需21M参数和3.4G MACs。相比之下，成熟的ViT基线DeiT-S [53], 达到了略差的准确率79.8%，并且需要35%更多的MACs（4.6G）。为了获得相似的准确率，MLP类模型ResMLP-S24[52] 需要43%更多的参数（30M）以及76%更多的计算（6.0G），但仅达到了79.4的准确率。即使与更改进的ViT和MLP类变体 [35, 57], 相比，PoolFormer仍然表现出更好的性能。具体来说，金字塔Transformer PVT-Medium获得了81.2的top-1准确率，参数为44M，MACs为6.7G，而PoolFormer-S36则达到了81.4，参数比PVT-Medium少30%（31M），MACs少25%（5.0G）。

此外，与 RSB-ResNet（“ResNet反击”） [59]相比，其中 ResNet [24]使用改进的训练程序在相同的 300 个周期内进行训练，PoolFormer 仍然表现更好。使用 $\sim$22M 参数/3.7G MACs，RSB-ResNet-34 [59] 获得 75.5 的准确率，而 PoolFormer-S24 可以获得 80.3。由于池化层的局部空间建模能力远不如神经卷积层，PoolFormer 的竞争性能只能归因于其通用架构 MetaFormer。

使用池化算子，每个标记均匀地聚合其附近标记的特征。因此，这是一种极其基本的 token 混合操作。然而，实验结果表明，即使使用这种令人尴尬的简单的 token 混合器，MetaFormer 仍然获得了极具竞争力的性能。图 3 清楚地表明，PoolFormer 超过了其他使用更少 MACs 和参数的模型。这一发现表明，通用架构 MetaFormer 实际上是在设计视觉模型时我们所需要的。通过采用 MetaFormer，可以保证派生的模型将具有实现合理性能的潜力。

4.2. 目标检测和实例分割

设置。我们在具有挑战性的 COCO 基准 [34] 上评估 PoolFormer，该基准包括 118K 个训练图像（train2017）和 5K 个验证图像（val2017）。模型在训练集上进行训练，并在验证集上报告性能。PoolFormer 被用作两个标准检测器的骨干网络，即 RetinaNet [33] 和 Mask R-CNN [23]。使用 ImageNet 预训练权重初始化骨干网络，并使用 Xavier [20] 初始化添加的层。采用 AdamW [29,37] 进行训练，初始学习率为 1×10^{-4}，批大小为 16。在 [23, 33], 之后，我们采用 $1\times$ 训练计划，即训练检测模型 12 个周期。训练图像被调整为短边为 800 像素，长边不超过 1,333 像素。对于测试，图像的短边也调整为 800 像素。该实现基于 mmdetection [34] 代码库，实验在 8 个 NVIDIA A100 GPUs 上运行。

该实现基于 mmdetection [4] 代码库，实验在 8 个 NVIDIA A100 GPUs 上运行。

结果。配备了RetinaNet进行目标检测，基于PoolFormer的模型始终优于其可比的ResNet对应模型，如表3所示。例如，PoolFormer-S12实现了36.2 AP，大幅超越了ResNet-18（31.8 AP）。对于基于Mask R-CNN进行目标检测和实例分割的模型，观察到类似的结果。例如，PoolFormer-S12大幅超越了ResNet-18（边界框AP 37.3 vs. 34.0，以及掩码AP 34.6 vs. 31.2）。总体而言，对于COCO目标检测和实例分割，PoolFormers实现了具有竞争力的性能，始终优于ResNet的对应模型。

4.3. 语义分割

设置。ADE20K [67], 是一个具有挑战性的场景解析基准，被选用来评估语义分割模型的性能。该数据集包括训练集和验证集中的20K和2K图像，涵盖150个细粒度语义类别。PoolFormers被评估为配备了语义FPN [30]的骨干网络。使用ImageNet-1K训练的检查点来初始化骨干网络，而Xavier [20] 被用于初始化其他新添加的层。常见的做法 [5, 30] 使用批大小为16训练模型80K次。为了加速训练，我们将批大小增加到32，并将迭代次数减少到40K。AdamW [29,37]被采用，初始学习率为 2×10^{-4}，该学习率将在多项式衰减调度中以0.9的幂次衰减。图像被调整大小并裁剪成 512×512 用于训练，并在测试时调整大小为512像素的短边。我们的实现基于mmsegmentation [10] 代码库，实验在8块NVIDIA A100 GPUs上运行。

结果。表4显示了使用FPN [30]的不同骨干网络的ADE20K语义分割性能。基于PoolFormer的模型始终优于使用基于CNN的ResNet [24] 和ResNeXt [62] 以及基于Transformer的PVT的模型。例如，PoolFormer-12在mIoU方面分别比ResNet-18和PVT-Tiny高37.1、4.3和1.5。

这些结果表明，我们的PoorFormer作为骨干网络可以在语义分割方面取得具有竞争力的性能，尽管它仅利用池化来基本上在标记之间传递信息。这进一步表明了MetaFormer的巨大潜力，并支持我们的观点，即MetaFormer实际上就是我们需要的。

4.4. 消融研究

消融研究实验在 ImageNet-1K [14] 上进行。表 5 报告了 PoolFormer 的消融研究。我们根据以下方面讨论消融。

Backbone	RetinaNet 1×							Mask R-CNN 1×						
	Params (M)	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L	Params (M)	AP^b	AP^b_{50}	AP^b_{75}	AP^m	AP^m_{50}	AP^m_{75}
ResNet-18 [24]	21.3	31.8	49.6	33.6	16.3	34.3	43.2	31.2	34.0	54.0	36.7	31.2	51.0	32.7
PoolFormer-S12	21.7	36.2	56.2	38.2	20.8	39.1	48.0	31.6	37.3	59.0	40.1	34.6	55.8	36.9
ResNet-50 [24]	37.7	36.3	55.3	38.6	19.3	40.0	48.8	44.2	38.0	58.6	41.4	34.4	55.1	36.7
PoolFormer-S24	31.1	38.9	59.7	41.3	23.3	42.1	51.8	41.0	40.1	62.2	43.4	37.0	59.1	39.6
ResNet-101 [24]	56.7	38.5	57.8	41.2	21.4	42.6	51.1	63.2	40.4	61.1	44.2	36.4	57.7	38.8
PoolFormer-S36	40.6	39.5	60.5	41.8	22.5	42.9	52.4	50.5	41.0	63.1	44.8	37.7	60.1	40.0

Table 3. **Performance of object detection using RetinaNet, and object detection and instance segmentation using Mask R-CNN on COCO `val2017` [34].** 1× training schedule (*i.e.*12 epochs) is used for training detection models. AP^b and AP^m represent bounding box AP and mask AP, respectively.

Backbone	Semantic FPN	
	Params (M)	mIoU (%)
ResNet-18 [24]	15.5	32.9
PVT-Tiny [57]	17.0	35.7
PoolFormer-S12	15.7	37.2
ResNet-50 [24]	28.5	36.7
PVT-Small [57]	28.2	39.8
PoolFormer-S24	23.2	40.3
ResNet-101 [24]	47.5	38.8
ResNeXt-101-32x4d [62]	47.1	39.7
PVT-Medium [57]	48.0	41.6
PoolFormer-S36	34.6	42.0
PVT-Large [57]	65.1	42.1
PoolFormer-M36	59.8	42.4
ResNeXt-101-64x4d [62]	86.4	40.2
PoolFormer-M48	77.1	42.7

Table 4. **Performance of Semantic segmentation on ADE20K [67] validation set.** All models are equipped with Semantic FPN [30].

Token mixers. Compared with Transformers, the main change made by PoolFormer is using simple pooling as a token mixer. We first conduct ablation for this operator by directly replacing pooling with identity mapping. Surprisingly, MetaFormer with identity mapping can still achieve 74.3% top-1 accuracy, supporting the claim that MetaFormer is actually what we need to guarantee reasonable performance.

Then the pooling is replaced with global random matrix $W_R \in \mathbb{R}^{N\times N}$ for each block. The matrix is initialized with random values from a uniform distribution on the interval [0, 1), and then Softmax is utilized to normalize each row. After random initialization, the matrix parameters are frozen and it conducts token mixing by $X' = W_R X$ where $X \in \mathbb{R}^{N\times C}$ are the input token features with the token length of N and channel dimension of C. The token mixer of random matrix introduces extra 21M frozen parameters for the S12 model since the token lengths are extremely large at the first stage. Even with such random token mixing method, the model can still achieve reasonable performance of 75.8% accuracy, 1.5% higher than that of identity mapping. It shows that MetaFormer can still work well even with random token mixing, not to say with other well-designed token mixers.

Further, pooling is replaced with Depthwise Convolution [9, 38] that has learnable parameters for spatial modeling. Not surprisingly, the derived model still achieve highly competitive performance with top-1 accuracy of 78.1%, 0.9% higher than PoolFormer-S12 due to its better local spatial modeling ability. Until now, we have specified multiple token mixers in Metaformer, and all resulted models keep promising results, well supporting the claim that MetaFormer is the key to guaranteeing models' competitiveness. Due to the simplicity of pooling, it is mainly utilized as a tool to demonstrate MetaFormer.

We test the effects of pooling size on PoolFormer. We observe similar performance when pooling sizes are 3, 5, and 7. However, when the pooling size increases to 9, there is an obvious performance drop of 0.5%. Thus, we adopt the default pooing size of 3 for PoolFormer.

Normalization. We modify Layer Normalization [1] into Modified Layer Normalization (MLN) that computes the mean and variance along token and channel dimensions compared with only channel dimension in vanilla Layer Normalization. The shape of learnable affine parameters of MLN keeps the same as that of Layer Normalization, *i.e.*, $\mathbb{R}^C$. MLN can be implemented with GroupNorm API in PyTorch by setting the group number as 1. See the appendix for details. We find PoolFormer prefers MLN with 0.7% or 0.8% higher than Layer Normalization or Batch Normalization. Thus, MLN is set as default for PoolFormer. When removing normalization, the model can not be trained to converge well, and its performance dramatically drops to only 46.1%.

Activation. We change GELU [25] to ReLU [41] or SiLU [18]. When ReLU is adopted for activation, an obvious performance drop of 0.8% is observed. For SiLU, its performance is almost the same as that of GELU. Thus, we still adopt GELU as default activation.

Other components. Besides token mixer and normalization discussed above, residual connection [24] and channel MLP [46, 47] are two other important components in MetaFormer. Without residual connection or channel MLP,

骨干网络	RetinaNet 1×							Mask R-CNN 1×						
	参数（M）	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L	参数（M）	AP^b	AP^b_{50}	AP^b_{75}	AP^m	AP^m_{50}	AP^m_{75}
ResNet-18 [24]	21.3	31.8	49.6	33.6	16.3	34.3	43.2	31.2	34.0	54.0	36.7	31.2	51.0	32.7
PoolFormer-S12	21.7	36.2	56.2	38.2	20.8	39.1	48.0	31.6	37.3	59.0	40.1	34.6	55.8	36.9
ResNet-50 [24]	37.7	36.3	55.3	38.6	19.3	40.0	48.8	44.2	38.0	58.6	41.4	34.4	55.1	36.7
PoolFormer-S24	31.1	38.9	59.7	41.3	23.3	42.1	51.8	41.0	40.1	62.2	43.4	37.0	59.1	39.6
ResNet-101 [24]	56.7	38.5	57.8	41.2	21.4	42.6	51.1	63.2	40.4	61.1	44.2	36.4	57.7	38.8
PoolFormer-S36	40.6	39.5	60.5	41.8	22.5	42.9	52.4	50.5	41.0	63.1	44.8	37.7	60.1	40.0

表3. 使用RetinaNet进行目标检测，以及使用Mask R-CNN在COCO val`2017` [34]上进行目标检测和实例分割的性能。 1× 训练计划（即12个周期）用于训练检测模型。 AP^b 和 AP^m 分别表示边界框AP和掩码AP。

骨干网络	语义FPN 参数	
	（M）	mIoU(%)
ResNet-18 [24]	15.5	32.9
PVT-Tiny [57]	17.0	35.7
PoolFormer-S12	15.7	37.2
ResNet-50 [24]	28.5	36.7
PVT-Small [57]	28.2	39.8
PoolFormer-S24	23.2	40.3
ResNet-101 [24]	47.5	38.8
ResNeXt-101-32x4d [62]	47.1	39.7
PVT-Medium [57]	48.0	41.6
PoolFormer-S36	34.6	42.0
PVT-Large [57]	65.1	42.1
PoolFormer-M36	59.8	42.4
ResNeXt-101-64x4d [62]	86.4	40.2
PoolFormer-M48	77.1	42.7

表4. 在ADE20K [67] 验证集上的语义分割性能。所有模型都配备了语义FPN [30]。

Token 混合器。与 Transformer 相比，PoolFormer 的主要变化是使用简单的池化作为 Token 混合器。我们首先通过直接用 identity mapping 替换池化来对此操作符进行消融。令人惊讶的是，使用 identity mapping 的 MetaFormer 仍然可以达到 74.3% 的 top-1 准确率，支持 MetaFormer 实际上就是我们需要的来保证合理性能的说法。

然后池化操作被全局随机矩阵 $W_R \in \mathbb{R}^{N\times N}$ 替代，每个块使用一个。该矩阵使用区间 [0, 1 上的均匀分布随机值初始化，然后使用 Softmax 对每一行进行归一化。随机初始化后，矩阵参数被冻结，并通过 $X' = W_R X$ 进行 token 混合，其中 $X \in \mathbb{R}^{N\times C}$ 是具有 N 长度和 C 通道维度的输入 token 特征。随机矩阵的 token 混合器为 S12 模型引入了额外的 21M 冻结参数，因为在第一阶段 token 长度非常大。即使使用这种随机 token 混合方法，模型仍然可以达到 75.8% 的准确率，比 identity mapping 高 1.5%。这表明即使使用随机 token 混合，MetaFormer 仍然可以很好地工作，更不用说其他精心设计的 token 混合器。

此外，池化被替换为具有空间建模可学习参数的深度卷积 [9, 38] 。毫不奇怪，推导出的模型仍然实现了高度有竞争力的性能，top-1准确率为78.1%，比 PoolFormer-S12高0.9%，这归因于其更好的局部空间建模能力。到目前为止，我们在Metaformer中指定了多个token混合器，所有结果模型都保持了有希望的结果，有力支持了MetaFormer是保证模型竞争力的关键这一说法。由于池化的简单性，它主要被用作展示MetaFormer的工具。

我们测试了池化大小对PoolFormer的影响。当池化大小为3、5和7时，我们观察到相似的性能。然而，当池化大小增加到9时，性能明显下降了0.5%。因此，我们采用PoolFormer的默认池化大小3。

归一化。我们修改了 Layer Normalization [1] 为改进层归一化（MLN），MLN 沿 token 和通道维度计算均值和方差，而 vanilla Layer Normalization 仅在通道维度上计算。MLN 的可学习仿射参数的形状与 Layer Normalization 相同，即 $\mathbb{R}^C$。MLN 可以通过在 PyTorch 中设置组数为 1 来使用 GroupNorm API 实现。详见附录。我们发现 PoolFormer 使用 MLN 比使用 Layer Normalization 或 Batch Normal- ization 高 0.7% 或 0.8%。因此，MLN 被设置为 PoolFormer 的默认选项。当移除归一化时，模型无法很好地训练收敛，其性能急剧下降至仅 46.1%。

激活。我们将 GELU [25]改为 ReLU [41] 或 SiLU[18]。当采用 ReLU 作为激活函数时，观察到明显的性能下降 0.8%。对于 SiLU，其性能与 GELU 几乎相同。因此，我们仍然采用 GELU 作为默认激活函数。

其他组件。除了上述讨论的 token 混合器 和 归一化之外，残差连接 [24]和 通道 MLP [46, 47] 是 MetaFormer 中的另外两个重要组件。没有残差连接或 通道 MLP,

Ablation	Variant	Params (M)	MACs (G)	Top-1 (%)
Baseline	None (PoolFormer-S12)	11.9	1.8	77.2
Token mixers	Pooling → Identity mapping	11.9	1.8	74.3
	Pooling → Global random matrix* (extra 21M frozen parameters)	11.9	3.3	75.8
	Pooling → Depthwise Convolution [9, 38]	11.9	1.8	78.1
	Pooling size 3 → 5	11.9	1.8	77.2
	Pooling size 3 → 7	11.9	1.8	77.1
	Pooling size 3 → 9	11.9	1.8	76.8
Normalization	Modified Layer Normalization† → Layer Normalization [1]	11.9	1.8	76.5
	Modified Layer Normalization† → Batch Normalization [28]	11.9	1.8	76.4
	Modified Layer Normalization† → None	11.9	1.8	46.1
Activation	GELU [25] → ReLU [41]	11.9	1.8	76.4
	GELU → SiLU [18]	11.9	1.8	77.2
Other components	Residual connection [25] → None	11.9	1.8	0.1
	Channel MLP → None	2.5	0.2	5.7
Hybrid Stages	[Pool, Pool, Pool, Pool] → [Pool, Pool, Pool, Attention]	14.0	1.9	78.3
	[Pool, Pool, Pool, Pool] → [Pool, Pool, Attention, Attention]	16.5	2.5	81.0
	[Pool, Pool, Pool, Pool] → [Pool, Pool, Pool, SpatialFC]	11.9	1.8	77.5
	[Pool, Pool, Pool, Pool] → [Pool, Pool, SpatialFC, SpatialFC]	12.2	1.9	77.9

Table 5. **Ablation for PoolFormer on ImageNet-1K classification benchmark.** PoolFormer-S12 is utilized as the baseline to conduct ablation study. The top-1 accuracy on the validation set is reported. *This token mixer utilizes global random matrix $W_R \in \mathbb{R}^{N\times N}$ (parameters are frozen after random initialization) to conduct token mixing by $X' = W_R X$ where $X \in \mathbb{R}^{N\times C}$ are input tokens with the token length of N and channel dimension of C. †Modified Layer Normalization (MLN) computes the mean and variance along token and channel dimensions compared with vanilla Layer Normalization only along channel dimension. MLN can be implemented with GroupNorm API in PyTorch by specifying the group number equal to 1. The numbers of MACs are counted by `fvcore` [19] library.

the model cannot converge and only achieves the accuracy of 0.1%/5.7%, proving the indispensability of these parts.

Hybrid stages. Among token mixers based on pooling, attention, and spatial MLP, the pooling-based one can handle much longer input sequences while attention and spatial MLP are good at capturing global information. Therefore, it is intuitive to stack MetaFormers with pooling in the bottom stages to handle long sequences and use attention or spatial MLP-based mixer in the top stages, considering the sequences have been largely shortened. Thus, we replace the token mixer pooling with attention or spatial FC [1] in the top one or two stages in PoolFormer. From Table 5, the hybrid models perform quite well. The variant with pooling in the bottom two stages and attention in the top two stages delivers highly competitive performance. It achieves 81.0% accuracy with only 16.5M parameters and 2.5G MACs. As a comparison, ResMLP-B24 needs 7.0× parameters (116M) and 9.2× MACs (23.0G) to achieve the same accuracy. These results indicate that combining pooling with other token mixers for MetaFormer may be a promising direction to further improve the performance.

5. Conclusion and future work

In this work, we abstracted the attention in Transformers as a token mixer, and the overall Transformer as a general architecture termed MetaFormer where the token mixer is not specified. Instead of focusing on specific token mixers, we point out that MetaFormer is actually what we need to guarantee achieving reasonable performance. To verify this, we deliberately specify token mixer as extremely simple pooling for MetaFormer. It is found that the derived PoolFormer model can achieve competitive performance on different vision tasks, which well supports that "MetaFormer is actually what you need for vision".

In the future, we will further evaluate PoolFormer under more different learning settings, such as self-supervised learning and transfer learning. Moreover, it is interesting to see whether PoolFormer still works on NLP tasks to further support the claim "MetaFormer is actually what you need" in the NLP domain. We hope that this work can inspire more future research devoted to improving the fundamental architecture MetaFormer instead of paying too much attention to the token mixer modules.

Acknowledgement

The authors would like to thank Quanhong Fu at Sea AI Lab for the help to improve the technical writing aspect of this paper. Weihao Yu would like to thank TPU Research Cloud (TRC) program and Google Cloud research credits for the support of partial computational resources. This project is in part supported by NUS Faculty Research Committee Grant (WBS: A-0009440-00-00). Shuicheng Yan and Xinchao Wang are the corresponding authors.

[1]Following [52], we use only one spatial fully connected layer as a token mixer, so we call it FC.

消融	变体	参数（M）	MACs（G）	Top-1（%）
基线	None (PoolFormer-S12)	11.9	1.8	77.2
Token mixers	池化 →identity mapping	11.9	1.8	74.3
	池化 →全局随机矩阵* (额外 21M 冻结参数)	11.9	3.3	75.8
	池化 → 深度卷积 [9, 38]	11.9	1.8	78.1
	池化大小 3 → 5	11.9	1.8	77.2
	池化大小 3 → 7	11.9	1.8	77.1
	池化大小 3 → 9	11.9	1.8	76.8
归一化	改进层归一化† →层归一化 [1]	11.9	1.8	76.5
	改进层归一化† →批量归一化 [28]	11.9	1.8	76.4
	改进层归一化† →None	11.9	1.8	46.1
激活	GELU [25] → ReLU [41]	11.9	1.8	76.4
	GELU →SiLU [18]	11.9	1.8	77.2
其他组件	残差连接 [25] → None	11.9	1.8	0.1
	通道 MLP → None	2.5	0.2	5.7
混合阶段	[池, 池, 池, 池] → [池, 池, 池, 注意力]	14.0	1.9	78.3
	[池, 池, 池, 池] → [池, 池, 注意力, 注意力]	16.5	2.5	81.0
	[池, 池, 池, 池] → [池, 池, 池, 空间FC]	11.9	1.8	77.5
	[池, 池, 池, 池] → [池, 池, 空间FC, 空间FC]	12.2	1.9	77.9

表 5. PoolFormer 在 ImageNet-1K 分类基准上的消融研究。PoolFormer-S12 被用作基线进行消融研究。报告了验证集上的 top-1 准确率。 *这个 token 混合器利用全局随机矩阵 $W_R \in \mathbb{R}^{N\times N}$（参数在随机初始化后冻结）进行 token 混合，通过 $X' = W_R X$，其中 $X \in \mathbb{R}^{N\times C}$ 是输入 token，其 token 长度为 N 且通道维度为 C。 †改进层归一化（MLN）沿 token 和通道维度计算均值和方差，而 vanilla Layer Normalization 仅沿通道维度计算。MLN 可以通过在 PyTorch 中指定组数等于 1 来使用 GroupNorm API 实现。MACs 的数量由 fvcore [19] 库统计。

模型无法收敛，并且仅达到 0.1%/5.7% 的准确率，证明了这些部分的不可或缺性。

混合阶段。在基于池、注意力和空间MLP的token混合器中，基于池的混合器可以处理更长的输入序列，而注意力和空间MLP擅长捕获全局信息。因此，直观地堆叠MetaFormer在底部阶段使用池来处理长序列，并在顶部阶段使用注意力或基于空间MLP的混合器，考虑到序列已经被大大缩短。因此，我们在PoolFormer的顶部一个或两个阶段中用注意力或空间FC[1] 替换了token混合器池。从表5可以看出，混合模型表现相当好。底部两个阶段使用池、顶部两个阶段使用注意力的变体实现了极具竞争力的性能。它仅使用16.5M参数和2.5G MACs就达到了81.0%的准确率。相比之下，ResMLP-B24需要 7.0× 参数（116M）和 9.2× MACs（23.0G）才能达到相同的准确率。这些结果表明，将池与其他token混合器结合用于MetaFormer可能是一个有前景的方向，以进一步提高性能。

5. 结论与未来工作

在这项工作中，我们将 Transformer 中的注意力抽象为一个 token 混合器，并将整体 Transformer 视为一个通用架构，称为 MetaFormer，其中 token 混合器未指定。我们不是专注于特定的 token 混合器，而是指出 MetaFormer 实际上是我们需要保证实现合理性能的任务。为了验证这一点，我们故意将 token 混合器指定为 MetaFormer 的极其简单的池化。发现派生的 Pool-Former 模型在不同视觉任务上可以实现具有竞争力的性能，这很好地支持了“MetaFormer 实际上是您需要用于视觉的任务”这一观点。

未来，我们将进一步在更多不同的学习设置下评估PoolFormer，例如自监督学习和迁移学习。此外，有趣的是观察PoolFormer是否仍然适用于NLP任务，以进一步支持“MetaFormer实际上是您需要的”这一NLP领域的声明。我们希望这项工作能够启发更多未来的研究，致力于改进MetaFormer的基本架构，而不是过多关注token混合器模块。

致谢

作者感谢Sea AI Lab的Fu全红在改进本文技术写作方面的帮助。Weihao Yu感谢TPU研究云（TRC）项目和Google Cloud研究学分对部分计算资源的支持。该项目部分由NUS学院研究委员会资助（WBS: A-0009440-00-00）。Shuicheng Yan和Xinchao Wang是通讯作者。

[1]遵循 [52]，我们仅使用一个空间全连接层作为token混合器，因此我们称其为FC。

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. 3, 5, 7, 8, 12

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. 2

[3] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision*, pages 213–229. Springer, 2020. 2

[4] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. MMDetection: Open mmlab detection toolbox and benchmark. *arXiv preprint arXiv:1906.07155*, 2019. 6

[5] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE transactions on pattern analysis and machine intelligence*, 40(4):834–848, 2017. 6

[6] Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. Generative pretraining from pixels. In *International Conference on Machine Learning*, pages 1691–1703. PMLR, 2020. 2

[7] Shoufa Chen, Enze Xie, Chongjian Ge, Ding Liang, and Ping Luo. Cyclemlp: A mlp-like architecture for dense prediction. *arXiv preprint arXiv:2107.10224*, 2021. 3

[8] Yunpeng Chen, Yannis Kalantidis, Jianshu Li, Shuicheng Yan, and Jiashi Feng. Aˆ 2-nets: Double attention networks. *Advances in Neural Information Processing Systems*, 31:352–361, 2018. 2

[9] François Chollet. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1251–1258, 2017. 7, 8

[10] MMSegmentation Contributors. MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark. https://github.com/open-mmlab/mmsegmentation, 2020. 6

[11] Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. Randaugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 702–703, 2020. 4

[12] Stéphane d'Ascoli, Hugo Touvron, Matthew Leavitt, Ari Morcos, Giulio Biroli, and Levent Sagun. Convit: Improving vision transformers with soft convolutional inductive biases. *arXiv preprint arXiv:2103.10697*, 2021. 2

[13] Stéphane D'Ascoli, Hugo Touvron, Matthew L Leavitt, Ari S Morcos, Giulio Biroli, and Levent Sagun. Convit: Improving vision transformers with soft convolutional inductive biases. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 2286–2296. PMLR, 18–24 Jul 2021. 2

[14] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 2, 4, 6

[15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT (1)*, 2019. 2

[16] Yihe Dong, Jean-Baptiste Cordonnier, and Andreas Loukas. Attention is not all you need: Pure attention loses rank doubly exponentially with depth. *arXiv preprint arXiv:2103.03404*, 2021. 3

[17] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020. 1, 2, 3, 5

[18] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018. 7, 8

[19] fvcore Contributors. fvcore. https://github.com/facebookresearch/fvcore, 2021. 4, 5, 8

[20] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010. 6

[21] Jianyuan Guo, Kai Han, Han Wu, Chang Xu, Yehui Tang, Chunjing Xu, and Yunhe Wang. Cmt: Convolutional neural networks meet vision transformers. *arXiv preprint arXiv:2107.06263*, 2021. 2

[22] Kai Han, An Xiao, Enhua Wu, Jianyuan Guo, Chunjing Xu, and Yunhe Wang. Transformer in transformer. *arXiv preprint arXiv:2103.00112*, 2021. 2

[23] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017. 6

[24] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 1, 4, 5, 6, 7, 12

参考文献

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. 3, 5, 7, 8, 12

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. 2

[3] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision*, pages 213–229. Springer, 2020. 2

[4] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. MMDetection: Open mmlab detection toolbox and benchmark. *arXiv preprint arXiv:1906.07155*, 2019. 6

[5] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE transactions on pattern analysis and machine intelligence*, 40(4):834–848, 2017. 6

[6] Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. Generative pretraining from pixels. In *International Conference on Machine Learning*, pages 1691–1703. PMLR, 2020. 2

[7] Shoufa Chen, Enze Xie, Chongjian Ge, Ding Liang, and Ping Luo. Cyclemlp: A mlp-like architecture for dense prediction. *arXiv preprint arXiv:2107.10224*, 2021. 3

[8] Yunpeng Chen, Yannis Kalantidis, Jianshu Li, Shuicheng Yan, and Jiashi Feng. Aˆ 2-nets: Double attention networks. *Advances in Neural Information Processing Systems*, 31:352–361, 2018. 2

[9] François Chollet. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1251–1258, 2017. 7, 8

[10] MMSegmentation Contributors. MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark. https://github.com/open-mmlab/mmsegmentation, 2020. 6

[11] Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. Randaugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 702–703, 2020. 4

[12] Stéphane d'Ascoli, Hugo Touvron, Matthew Leavitt, Ari Morcos, Giulio Biroli, and Levent Sagun. Convit: Improving vision transformers with soft convolutional inductive biases. *arXiv preprint arXiv:2103.10697*, 2021. 2

[13] Stéphane D'Ascoli, Hugo Touvron, Matthew L Leavitt, Ari S Morcos, Giulio Biroli, and Levent Sagun. Convit: Improving vision transformers with soft convolutional inductive biases. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 2286–2296. PMLR, 18–24 Jul 2021. 2

[14] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 2, 4, 6

[15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT (1)*, 2019. 2

[16] Yihe Dong, Jean-Baptiste Cordonnier, and Andreas Loukas. Attention is not all you need: Pure attention loses rank doubly exponentially with depth. *arXiv preprint arXiv:2103.03404*, 2021. 3

[17] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020. 1, 2, 3, 5

[18] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018. 7, 8

[19] fvcore Contributors. fvcore. https://github.com/facebookresearch/fvcore, 2021. 4, 5, 8

[20] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010. 6

[21] Jianyuan Guo, Kai Han, Han Wu, Chang Xu, Yehui Tang, Chunjing Xu, and Yunhe Wang. Cmt: Convolutional neural networks meet vision transformers. *arXiv preprint arXiv:2107.06263*, 2021. 2

[22] Kai Han, An Xiao, Enhua Wu, Jianyuan Guo, Chunjing Xu, and Yunhe Wang. Transformer in transformer. *arXiv preprint arXiv:2103.00112*, 2021. 2

[23] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017. 6

[24] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 1, 4, 5, 6, 7, 12

[25] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016. 3, 7, 8

[26] Qibin Hou, Zihang Jiang, Li Yuan, Ming-Ming Cheng, Shuicheng Yan, and Jiashi Feng. Vision permutator: A permutable mlp-like architecture for visual recognition. *arXiv preprint arXiv:2106.12368*, 2021. 2, 3

[27] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q Weinberger. Deep networks with stochastic depth. In *European conference on computer vision*, pages 646–661. Springer, 2016. 4, 12

[28] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. PMLR, 2015. 3, 8

[29] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 4, 6

[30] Alexander Kirillov, Ross Girshick, Kaiming He, and Piotr Dollár. Panoptic feature pyramid networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6399–6408, 2019. 6, 7

[31] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25:1097–1105, 2012. 4

[32] James Lee-Thorp, Joshua Ainslie, Ilya Eckstein, and Santiago Ontanon. Fnet: Mixing tokens with fourier transforms. *arXiv preprint arXiv:2105.03824*, 2021. 2

[33] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017. 6

[34] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014. 2, 6, 7

[35] Hanxiao Liu, Zihang Dai, David R So, and Quoc V Le. Pay attention to mlps. *arXiv preprint arXiv:2105.08050*, 2021. 2, 5, 6

[36] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10012–10022, October 2021. 2, 4, 5

[37] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2018. 4, 6

[38] Franck Mamalet and Christophe Garcia. Simplifying convnets for fast learning. In *International Conference on Artificial Neural Networks*, pages 58–65. Springer, 2012. 7, 8

[39] André Martins, António Farinhas, Marcos Treviso, Vlad Niculae, Pedro Aguiar, and Mario Figueiredo. Sparse and continuous attention mechanisms. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 20989–21001. Curran Associates, Inc., 2020. 2

[40] Pedro Henrique Martins, Zita Marinho, and André FT Martins. ∞-former: Infinite memory transformer. *arXiv preprint arXiv:2109.00301*, 2021. 2

[41] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Icml*, 2010. 3, 7, 8

[42] Namuk Park and Songkuk Kim. How do vision transformers work? In *International Conference on Learning Representations*, 2022. 3

[43] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *arXiv preprint arXiv:2108.08810*, 2021. 3

[44] Prajit Ramachandran, Niki Parmar, Ashish Vaswani, Irwan Bello, Anselm Levskaya, and Jon Shlens. Stand-alone self-attention in vision models. *Advances in Neural Information Processing Systems*, 32, 2019. 2

[45] Yongming Rao, Wenliang Zhao, Zheng Zhu, Jiwen Lu, and Jie Zhou. Global filter networks for image classification. *arXiv preprint arXiv:2107.00645*, 2021. 2

[46] Frank Rosenblatt. Principles of neurodynamics. perceptrons and the theory of brain mechanisms. Technical report, Cornell Aeronautical Lab Inc Buffalo NY, 1961. 7

[47] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning internal representations by error propagation. Technical report, California Univ San Diego La Jolla Inst for Cognitive Science, 1985. 7

[48] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*, pages 618–626, 2017. 12, 14

[49] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. 4

[50] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016. 4

[51] Ilya Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, et al. Mlp-mixer: An all-mlp architecture for vision. *arXiv preprint arXiv:2105.01601*, 2021. 1, 2, 3, 5

[52] Hugo Touvron, Piotr Bojanowski, Mathilde Caron, Matthieu Cord, Alaaeldin El-Nouby, Edouard Grave, Gautier Izacard, Armand Joulin, Gabriel Synnaeve, Jakob Verbeek, et al. Resmlp: Feedforward networks for image classification with data-efficient training. *arXiv preprint arXiv:2105.03404*, 2021. 1, 2, 3, 5, 6, 8, 12, 14

[25] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016. 3, 7, 8

[26] Qibin Hou, Zihang Jiang, Li Yuan, Ming-Ming Cheng, Shuicheng Yan, and Jiashi Feng. Vision permutator: A permutable mlp-like architecture for visual recognition. *arXiv preprint arXiv:2106.12368*, 2021. 2, 3

[27] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q Weinberger. Deep networks with stochastic depth. In *European conference on computer vision*, pages 646–661. Springer, 2016. 4, 12

[28] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. PMLR, 2015. 3, 8

[29] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 4, 6

[30] Alexander Kirillov, Ross Girshick, Kaiming He, and Piotr Dollár. Panoptic feature pyramid networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6399–6408, 2019. 6, 7

[31] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25:1097–1105, 2012. 4

[32] James Lee-Thorp, Joshua Ainslie, Ilya Eckstein, and Santiago Ontanon. Fnet: Mixing tokens with fourier transforms. *arXiv preprint arXiv:2105.03824*, 2021. 2

[33] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017. 6

[34] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014. 2, 6, 7

[35] Hanxiao Liu, Zihang Dai, David R So, and Quoc V Le. Pay attention to mlps. *arXiv preprint arXiv:2105.08050*, 2021. 2, 5, 6

[36] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10012–10022, October 2021. 2, 4, 5

[37] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2018. 4, 6

[38] Franck Mamalet and Christophe Garcia. Simplifying convnets for fast learning. In *International Conference on Artificial Neural Networks*, pages 58–65. Springer, 2012. 7, 8

[39] André Martins, António Farinhas, Marcos Treviso, Vlad Niculae, Pedro Aguiar, and Mario Figueiredo. Sparse and continuous attention mechanisms. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 20989–21001. Curran Associates, Inc., 2020. 2

[40] Pedro Henrique Martins, Zita Marinho, and André FT Martins. ∞-former: Infinite memory transformer. *arXiv preprint arXiv:2109.00301*, 2021. 2

[41] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Icml*, 2010. 3, 7, 8

[42] Namuk Park and Songkuk Kim. How do vision transformers work? In *International Conference on Learning Representations*, 2022. 3

[43] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *arXiv preprint arXiv:2108.08810*, 2021. 3

[44] Prajit Ramachandran, Niki Parmar, Ashish Vaswani, Irwan Bello, Anselm Levskaya, and Jon Shlens. Stand-alone self-attention in vision models. *Advances in Neural Information Processing Systems*, 32, 2019. 2

[45] Yongming Rao, Wenliang Zhao, Zheng Zhu, Jiwen Lu, and Jie Zhou. Global filter networks for image classification. *arXiv preprint arXiv:2107.00645*, 2021. 2

[46] Frank Rosenblatt. Principles of neurodynamics. perceptrons and the theory of brain mechanisms. Technical report, Cornell Aeronautical Lab Inc Buffalo NY, 1961. 7

[47] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning internal representations by error propagation. Technical report, California Univ San Diego La Jolla Inst for Cognitive Science, 1985. 7

[48] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*, pages 618–626, 2017. 12, 14

[49] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. 4

[50] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016. 4

[51] Ilya Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, et al. Mlp-mixer: An all-mlp architecture for vision. *arXiv preprint arXiv:2105.01601*, 2021. 1, 2, 3, 5

[52] Hugo Touvron, Piotr Bojanowski, Mathilde Caron, Matthieu Cord, Alaaeldin El-Nouby, Edouard Grave, Gautier Izacard, Armand Joulin, Gabriel Synnaeve, Jakob Verbeek, et al. Resmlp: Feedforward networks for image classification with data-efficient training. *arXiv preprint arXiv:2105.03404*, 2021. 1, 2, 3, 5, 6, 8, 12, 14

[53] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning*, pages 10347–10357. PMLR, 2021. 1, 2, 4, 5, 6, 12, 14

[54] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. *arXiv preprint arXiv:2103.17239*, 2021. 4

[55] Ashish Vaswani, Prajit Ramachandran, Aravind Srinivas, Niki Parmar, Blake Hechtman, and Jonathon Shlens. Scaling local self-attention for parameter efficient visual backbones. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12894–12904, 2021. 2

[56] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017. 1, 2, 3

[57] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 568–578, October 2021. 2, 4, 5, 6, 7

[58] Ross Wightman. Pytorch image models. `https://github.com/rwightman/pytorch-image-models`, 2019. 5

[59] Ross Wightman, Hugo Touvron, and Hervé Jégou. Resnet strikes back: An improved training procedure in timm. *arXiv preprint arXiv:2110.00476*, 2021. 1, 5, 6, 12, 14

[60] Haiping Wu, Bin Xiao, Noel Codella, Mengchen Liu, Xiyang Dai, Lu Yuan, and Lei Zhang. Cvt: Introducing convolutions to vision transformers. *arXiv preprint arXiv:2103.15808*, 2021. 2

[61] Kan Wu, Houwen Peng, Minghao Chen, Jianlong Fu, and Hongyang Chao. Rethinking and improving relative position encoding for vision transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10033–10041, 2021. 2

[62] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1492–1500, 2017. 6, 7

[63] Li Yuan, Yunpeng Chen, Tao Wang, Weihao Yu, Yujun Shi, Zi-Hang Jiang, Francis E.H. Tay, Jiashi Feng, and Shuicheng Yan. Tokens-to-token vit: Training vision transformers from scratch on imagenet. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 558–567, October 2021. 2, 3

[64] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6023–6032, 2019. 4

[65] Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations*, 2018. 4

[66] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 13001–13008, 2020. 4

[67] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ade20k dataset. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 633–641, 2017. 2, 6, 7

[68] Daquan Zhou, Yujun Shi, Bingyi Kang, Weihao Yu, Zihang Jiang, Yuan Li, Xiaojie Jin, Qibin Hou, and Jiashi Feng. Refiner: Refining self-attention for vision transformers. *arXiv preprint arXiv:2106.03714*, 2021. 2, 3

[53] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning*, pages 10347–10357. PMLR, 2021. 1, 2, 4, 5, 6, 12, 14

[54] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. *arXiv preprint arXiv:2103.17239*, 2021. 4

[55] Ashish Vaswani, Prajit Ramachandran, Aravind Srinivas, Niki Parmar, Blake Hechtman, and Jonathon Shlens. Scaling local self-attention for parameter efficient visual backbones. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12894–12904, 2021. 2

[56] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017. 1, 2, 3

[57] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 568–578, October 2021. 2, 4, 5, 6, 7

[58] Ross Wightman. Pytorch image models. `https://github.com/rwightman/pytorch-image-models`, 2019. 5

[59] Ross Wightman, Hugo Touvron, and Hervé Jégou. Resnet strikes back: An improved training procedure in timm. *arXiv preprint arXiv:2110.00476*, 2021. 1, 5, 6, 12, 14

[60] Haiping Wu, Bin Xiao, Noel Codella, Mengchen Liu, Xiyang Dai, Lu Yuan, and Lei Zhang. Cvt: Introducing convolutions to vision transformers. *arXiv preprint arXiv:2103.15808*, 2021. 2

[61] Kan Wu, Houwen Peng, Minghao Chen, Jianlong Fu, and Hongyang Chao. Rethinking and improving relative position encoding for vision transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10033–10041, 2021. 2

[62] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1492–1500, 2017. 6, 7

[63] Li Yuan, Yunpeng Chen, Tao Wang, Weihao Yu, Yujun Shi, Zi-Hang Jiang, Francis E.H. Tay, Jiashi Feng, and Shuicheng Yan. Tokens-to-token vit: Training vision transformers from scratch on imagenet. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 558–567, October 2021. 2, 3

[64] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6023–6032, 2019. 4

[65] Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations*, 2018. 4

[66] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 13001–13008, 2020. 4

[67] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ade20k dataset. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 633–641, 2017. 2, 6, 7

[68] Daquan Zhou, Yujun Shi, Bingyi Kang, Weihao Yu, Zihang Jiang, Yuan Li, Xiaojie Jin, Qibin Hou, and Jiashi Feng. Refiner: Refining self-attention for vision transformers. *arXiv preprint arXiv:2106.03714*, 2021. 2, 3

A. Detailed hyper-parameters on ImageNet-1K

PoolFormer. On ImageNet-1K classification benchmark, we utilize the hyper-parameters shown in Table 6 to train models in our paper. Based on the relation between batch size and learning rate in Table 6, we set the batch size as 4096 and learning rate as 4×10^{-3}. For stochastic depth, following the original paper [27], we linearly increase the probability of dropping a layer from 0.0 for the bottom block to d_r for the top block.

Hybrid Models. We use the hyper-parameters for all models except for the hybrid models with token mixers of pooling and attention. For these hybrid models, we find that they achieve much better performances by setting batch size as 1024, learning rate as 10^{-3}, and normalization as Layer Normalization [1].

B. Training for longer epochs

In our paper, PoolFormer models are trained for the default 300 epochs on ImageNet-1K. For DeiT [53]/ResMLP [52], it is observed that the performance saturates after 400/800 epochs. Thus, we also conduct the experiments of training longer for PoolFormer-S12 and the results are shown in Table 7. We observe that PoolFormer-S12 obtains saturated performance after around 2000 epochs with a top-1 accuracy improvement of 1.8%. However, for fair comparison with other ViT/MLP-like models, we still train PoolFormers for 300 epochs by default.

C. Qualitative results

We use Grad-CAM [48] to visualize the results of different models trained on ImageNet-1K. We find that although ResMLP [52] also activates some irrelevant parts, all models can locate the semantic objects. The activation parts of DeiT [53] and ResMLP [52] in the maps are more scattered, while those of RSB-ResNet [24, 59] and PoolFormer are more gathered.

D. Comparison between Layer Normalization and Modified Layer Normalization

We modify Layer Normalization [1] into Modified Layer Normalization (MNN). It computes the mean and variance along spatial and channel dimensions, compared with only channel dimension in vanilla Layer Normalization. The shape of learnable affine parameters of MLN keeps the same as that of Layer Normalization, *i.e.*, $\mathbb{R}^C$. MLN can be implemented with GroupNorm API in PyTorch by setting the group number as 1. The comparison details are shown in Algorithm 2.

E. Code in PyTorch

We provide the PyTorch-like code in Algorithm 3 associated with the modules used in the PoolFormer block. Algorithm 4 further shows the PoolFormer block built with these modules.

A. ImageNet-1K的详细超参数

PoolFormer. 在ImageNet-1K分类基准上，我们使用表6中显示的超参数来训练我们论文中的模型。根据表6中批大小和学习率的关系，我们将批大小设置为4096，学习率设置为 4×10^{-3}。对于随机深度，遵循原始论文 [27], ，我们将丢弃一层层的概率从底部块的0.0线性增加到顶部块的 d_r 。

混合模型。我们使用除具有池化和注意力token混合器的混合模型之外的所有模型的超参数。对于这些混合模型，我们发现将批大小设置为1024、学习率设置为 10^{-3}，并将归一化设置为层归一化 [1]时，性能会显著提高。

B. 训练更长时间的周期

在我们的论文中，PoolFormer模型在ImageNet-1K上默认训练300个周期。对于DeiT [53]/ResMLP[52], ，观察到性能在400/800个周期后饱和。因此，我们还对PoolFormer-S12进行了更长时间的训练实验，结果如表7所示。我们观察到PoolFormer-S12在大约2000个周期后达到饱和性能，top-1准确率提高了1.8%。然而，为了与其他ViT/MLP类模型进行公平比较，我们仍然默认训练PoolFormer 300个周期。

C. 定性结果

我们使用 Grad-CAM [48] 来可视化在ImageNet-1K 上训练的不同模型的结果。我们发现，尽管 ResMLP [52] 也激活了一些无关部分，但所有模型都能定位到语义对象。DeiT [53] 和 ResMLP [52] 在图中的激活部分更分散，而 RSB-ResNet [24, 59] 和 PoolFormer 的激活部分更聚集。

D. Layer Normalization 与 Modified Layer Normalization 的比较

我们修改了层归一化 [1] 为改进层归一化 (MNN)。它在空间和通道维度上计算均值和方差，而 vanilla Layer Normalization 仅在通道维度上计算。MLN 的可学习仿射参数的形状与 Layer Normalization 相同，即 $\mathbb{R}^C$。MLN 可以通过在 PyTorch 中将组数设置为 1 来使用 GroupNorm API 实现。比较的详细信息显示在算法2中。

E. PyTorch 中的代码

我们提供了与 PoolFormer 模块中使用的模块相关的 PyTorch 代码（算法3）。算法4进一步展示了使用这些模块构建的 PoolFormer 模块。

	PoolFormer				
	S12	S24	S36	M36	M48
Peak drop rate of stoch. depth d_r	0.1	0.1	0.2	0.3	0.4
LayerScale initialization ϵ	10^{-5}	10^{-5}	10^{-6}	10^{-6}	10^{-6}
Data augmentation	AutoAugment				
Repeated Augmentation	off				
Input resolution	224				
Epochs	300				
Warmup epochs	5				
Hidden dropout	0				
GeLU dropout	0				
Classification dropout	0				
Random erasing prob	0.25				
EMA decay	0				
Cutmix α	1.0				
Mixup α	0.8				
Cutmix-Mixup switch prob	0.5				
Label smoothing	0.1				
Relation between peak learning rate and batch size	$\text{lr} = \frac{\text{batch\_size}}{1024} \times 10^{-3}$				
Batch size used in the paper	4096				
Peak learning rate used in the paper	4×10^{-4}				
Learning rate decay	cosine				
Optimizer	AdamW				
Adam ϵ	1e-8				
Adam (β_1, β_2)	(0.9, 0.999)				
Weight decay	0.05				
Gradient clipping	None				

Table 6. **Hyper-parameters for image classification on ImageNet-1K**

	PoolFormer				
	S12	S24	S36	M36	M48
随机深度 d_r的峰值下降率	0.1	0.1	0.2	0.3	0.4
LayerScale初始化 ϵ	10^{-5}	10^{-5}	10^{-6}	10^{-6}	10^{-6}
数据增强	AutoAugment				
重复增强	off				
输入分辨率	224				
周期	300				
预热周期	5				
隐藏 dropout	0				
GeLU dropout	0				
分类 dropout	0				
随机擦除概率	0.25				
EMA 衰减	0				
Cutmix α	1.0				
Mixup α	0.8				
Cutmix-Mixup 切换概率	0.5				
标签平滑	0.1				
峰值学习率与批大小之间的关系	$\text{lr} = \frac{\text{批大小}}{1024} \times 10^{-3}$				
峰值学习率与批大小之间的关系					
论文中使用的批大小	4096				
论文中使用的峰值学习率	4×10^{-4}				
学习率衰减	余弦				
优化器	AdamW				
Adam ϵ	1e-8				
Adam (β_1, β_2)	(0.9, 0.999)				
权重衰减	0.05				
梯度裁剪	None				

表6. 在ImageNet-1K上进行图像分类的超参数

# Epochs	300 (default)	400	500	1000	1500	2000	2500	3000
PoolFormer-S12	77.2	77.5	77.9	78.4	78.6	78.8	78.8	78.8

Table 7. **Performance of PoolFormer trained for different numbers of epochs**.

# 周期	300 (默认)	400	500	1000	1500	2000	2500	3000
PoolFormer-S12	77.2	77.5	77.9	78.4	78.6	78.8	78.8	78.8

表 7. PoolFormer 在不同周期数训练后的性能表现。

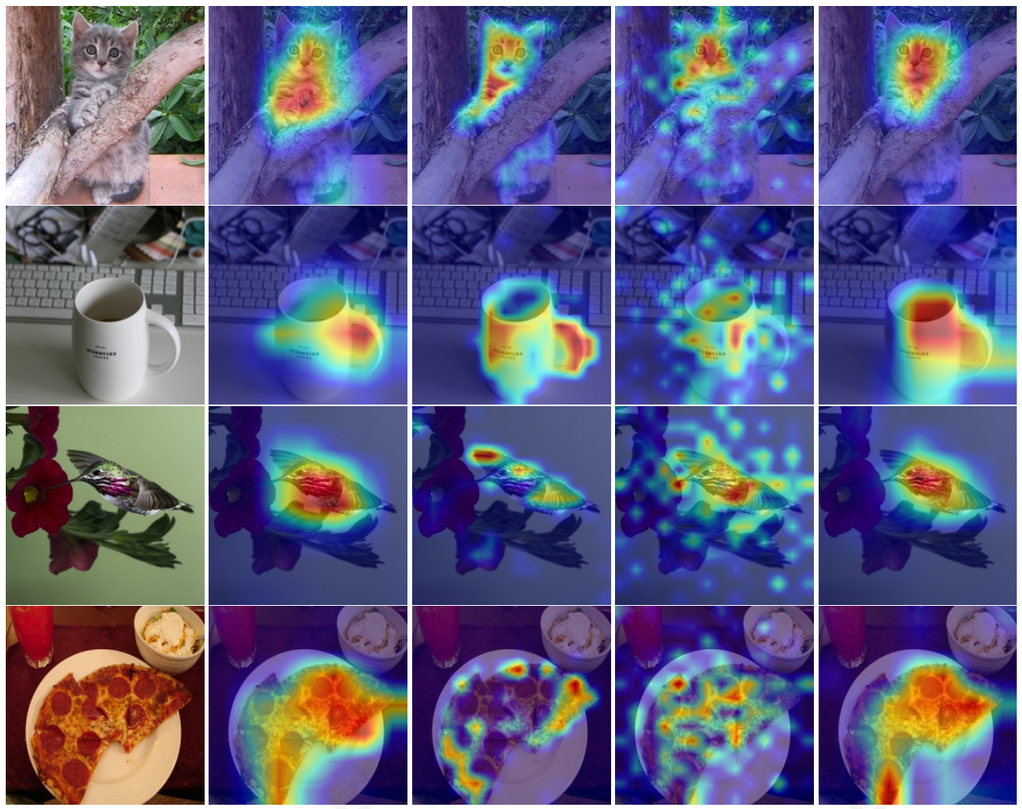


Figure 4. Grad-CAM [48] activation maps of the models trained on ImageNet-1K. The visualized images are from validation set.

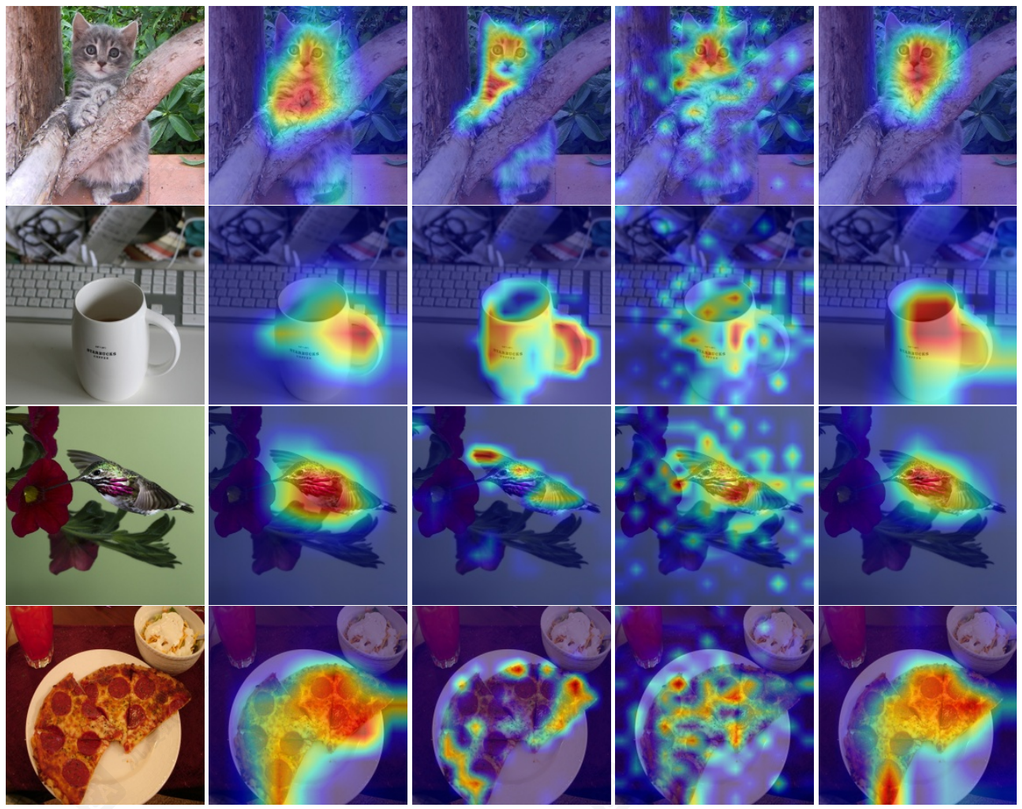


图4。在ImageNet-1K上训练的模型的Grad-CAM [48] 激活图。可视化图像来自验证集。

Algorithm 2 Comparison between Layer Normalization and Modified Layer Normalization, PyTorch-like Code

```
import torch.nn as nn


class LayerNormChannel(nn.Module):
    """
    Vanilla Layer Normalization normalizes vectors along channel dimension.
    Input: tensor in shape [B, C, H, W].
    """
    def __init__(self, num_channels, eps=1e-05):
        super().__init__()
        # The shape of learnable affine parameters is [num_channels, ].
        self.weight = nn.Parameter(torch.ones(num_channels))
        self.bias = nn.Parameter(torch.zeros(num_channels))
        self.eps = eps

    def forward(self, x):
        u = x.mean(1, keepdim=True) # Compute the means along channel dimension.
        s = (x - u).pow(2).mean(1, keepdim=True) # Compute the variances along channel dimension.
        x = (x - u) / torch.sqrt(s + self.eps)
        x = self.weight.unsqueeze(-1).unsqueeze(-1) * x \
            + self.bias.unsqueeze(-1).unsqueeze(-1)
        return x


class ModifiedLayerNorm(nn.Module):
    """
    Modified Layer Normalization normalizes vectors along channel dimension and spatial dimensions.
    Input: tensor in shape [B, C, H, W]
    """
    def __init__(self, num_channels, eps=1e-05):
        super().__init__()
        # The shape of learnable affine parameters is also [num_channels, ], keeping the same as vanilla Layer
            Normalization.
        self.weight = nn.Parameter(torch.ones(num_channels))
        self.bias = nn.Parameter(torch.zeros(num_channels))
        self.eps = eps

    def forward(self, x):
        u = x.mean([1, 2, 3], keepdim=True) # Compute the mean along channel dimension and spatial dimensions.
        s = (x - u).pow(2).mean([1, 2, 3], keepdim=True) # Compute the variance along channel dimension and
            spatial dimensions.
        x = (x - u) / torch.sqrt(s + self.eps)
        x = self.weight.unsqueeze(-1).unsqueeze(-1) * x \
            + self.bias.unsqueeze(-1).unsqueeze(-1)
        return x


# Modified Layer Normalization can also be implemented using GroupNorm API in PyTorch by setting the group
    number as 1.
class ModifiedLayerNorm(nn.GroupNorm):
    """
    Modified Layer Normalization implemented by Group Normalization with 1 group.
    Input: tensor in shape [B, C, H, W]
    """
    def __init__(self, num_channels, **kwargs):
        super().__init__(1, num_channels, **kwargs)
```

算法2 对比 层归一化 和 改进层归一化，类似PyTorch的代码

```
import torch.nn as nn


class LayerNormChannel(nn.Module):
    """
    Vanilla Layer Normalization normalizes vectors along channel dimension.
    Input: tensor in shape [B, C, H, W].
    """
    def __init__(self, num_channels, eps=1e-05):
        super().__init__()
        # The shape of learnable affine parameters is [num_channels, ].
        self.weight = nn.Parameter(torch.ones(num_channels))
        self.bias = nn.Parameter(torch.zeros(num_channels))
        self.eps = eps

    def forward(self, x):
        u = x.mean(1, keepdim=True) # Compute the means along channel dimension.
        s = (x - u).pow(2).mean(1, keepdim=True) # Compute the variances along channel dimension.
        x = (x - u) / torch.sqrt(s + self.eps)
        x = self.weight.unsqueeze(-1).unsqueeze(-1) * x \
            + self.bias.unsqueeze(-1).unsqueeze(-1)
        return x


class ModifiedLayerNorm(nn.Module):
    """
    Modified Layer Normalization normalizes vectors along channel dimension and spatial dimensions.
    Input: tensor in shape [B, C, H, W]
    """
    def __init__(self, num_channels, eps=1e-05):
        super().__init__()
        # The shape of learnable affine parameters is also [num_channels, ], keeping the same as vanilla Layer
            Normalization.
        self.weight = nn.Parameter(torch.ones(num_channels))
        self.bias = nn.Parameter(torch.zeros(num_channels))
        self.eps = eps

    def forward(self, x):
        u = x.mean([1, 2, 3], keepdim=True) # Compute the mean along channel dimension and spatial dimensions.
        s = (x - u).pow(2).mean([1, 2, 3], keepdim=True) # Compute the variance along channel dimension and
            spatial dimensions.
        x = (x - u) / torch.sqrt(s + self.eps)
        x = self.weight.unsqueeze(-1).unsqueeze(-1) * x \
            + self.bias.unsqueeze(-1).unsqueeze(-1)
        return x


# Modified Layer Normalization can also be implemented using GroupNorm API in PyTorch by setting the group
    number as 1.
class ModifiedLayerNorm(nn.GroupNorm):
    """
    Modified Layer Normalization implemented by Group Normalization with 1 group.
    Input: tensor in shape [B, C, H, W]
    """
    def __init__(self, num_channels, **kwargs):
        super().__init__(1, num_channels, **kwargs)
```

Algorithm 3 Modules for PoolFormer block, PyTorch-like Code

```
import torch.nn as nn


class ModifiedLayerNorm(nn.GroupNorm):
   """
   Modified Layer Normalization implemented by Group Normalization with 1 group.
   Input: tensor in shape [B, C, H, W]
   """
   def __init__(self, num_channels, **kwargs):
      super().__init__(1, num_channels, **kwargs)


class Pooling(nn.Module):
   """
   Implementation of pooling for PoolFormer
   --pool_size: pooling size
   Input: tensor with shape [B, C, H, W]
   """
   def __init__(self, pool_size=3):
      super().__init__()
      self.pool = nn.AvgPool2d(
         pool_size, stride=1, padding=pool_size//2, count_include_pad=False)

   def forward(self, x):
      # Subtraction of the input itself is added
      # since the block already has a residual connection.
      return self.pool(x) - x


class Mlp(nn.Module):
   """
   Implementation of MLP with 1*1 convolutions.
   Input: tensor with shape [B, C, H, W]
   """
   def __init__(self, in_features, hidden_features=None,
                out_features=None, act_layer=nn.GELU, drop=0.):
      super().__init__()
      out_features = out_features or in_features
      hidden_features = hidden_features or in_features
      self.fc1 = nn.Conv2d(in_features, hidden_features, 1)
      self.act = act_layer()
      self.fc2 = nn.Conv2d(hidden_features, out_features, 1)
      self.drop = nn.Dropout(drop)
      self.apply(self._init_weights)

   def _init_weights(self, m):
      if isinstance(m, nn.Conv2d):
         trunc_normal_(m.weight, std=.02)
         if m.bias is not None:
            nn.init.constant_(m.bias, 0)

   def forward(self, x):
      x = self.fc1(x)
      x = self.act(x)
      x = self.drop(x)
      x = self.fc2(x)
      x = self.drop(x)
      return x
```

算法3模块用于PoolFormer模块，类似PyTorch的代码

```
import torch.nn as nn


class ModifiedLayerNorm(nn.GroupNorm):
   """
   Modified Layer Normalization implemented by Group Normalization with 1 group.
   Input: tensor in shape [B, C, H, W]
   """
   def __init__(self, num_channels, **kwargs):
      super().__init__(1, num_channels, **kwargs)


class Pooling(nn.Module):
   """
   Implementation of pooling for PoolFormer
   --pool_size: pooling size
   Input: tensor with shape [B, C, H, W]
   """
   def __init__(self, pool_size=3):
      super().__init__()
      self.pool = nn.AvgPool2d(
         pool_size, stride=1, padding=pool_size//2, count_include_pad=False)

   def forward(self, x):
      # Subtraction of the input itself is added
      # since the block already has a residual connection.
      return self.pool(x) - x


class Mlp(nn.Module):
   """
   Implementation of MLP with 1*1 convolutions.
   Input: tensor with shape [B, C, H, W]
   """
   def __init__(self, in_features, hidden_features=None,
                out_features=None, act_layer=nn.GELU, drop=0.):
      super().__init__()
      out_features = out_features or in_features
      hidden_features = hidden_features or in_features
      self.fc1 = nn.Conv2d(in_features, hidden_features, 1)
      self.act = act_layer()
      self.fc2 = nn.Conv2d(hidden_features, out_features, 1)
      self.drop = nn.Dropout(drop)
      self.apply(self._init_weights)

   def _init_weights(self, m):
      if isinstance(m, nn.Conv2d):
         trunc_normal_(m.weight, std=.02)
         if m.bias is not None:
            nn.init.constant_(m.bias, 0)

   def forward(self, x):
      x = self.fc1(x)
      x = self.act(x)
      x = self.drop(x)
      x = self.fc2(x)
      x = self.drop(x)
      return x
```

Algorithm 4 PoolFormer block, PyTorch-like Code

```
import torch.nn as nn

class PoolFormerBlock(nn.Module):
    """
    Implementation of one PoolFormer block.
    --dim: embedding dim
    --pool_size: pooling size
    --mlp_ratio: mlp expansion ratio
    --act_layer: activation
    --norm_layer: normalization
    --drop: dropout rate
    --drop path: Stochastic Depth,
        refer to https://arxiv.org/abs/1603.09382
    --use_layer_scale, --layer_scale_init_value: LayerScale,
        refer to https://arxiv.org/abs/2103.17239
    """
    def __init__(self, dim, pool_size=3, mlp_ratio=4.,
                 act_layer=nn.GELU, norm_layer=ModifiedLayerNorm,
                 drop=0., drop_path=0.,
                 use_layer_scale=True, layer_scale_init_value=1e-5):

        super().__init__()

        self.norm1 = norm_layer(dim)
        self.token_mixer = Pooling(pool_size=pool_size)
        self.norm2 = norm_layer(dim)
        mlp_hidden_dim = int(dim * mlp_ratio)
        self.mlp = Mlp(in_features=dim, hidden_features=mlp_hidden_dim,
                       act_layer=act_layer, drop=drop)

        # The following two techniques are useful to train deep PoolFormers.
        self.drop_path = DropPath(drop_path) if drop_path > 0. \
            else nn.Identity()
        self.use_layer_scale = use_layer_scale
        if use_layer_scale:
            self.layer_scale_1 = nn.Parameter(
                layer_scale_init_value * torch.ones(dim), requires_grad=True)
            self.layer_scale_2 = nn.Parameter(
                layer_scale_init_value * torch.ones(dim), requires_grad=True)

    def forward(self, x):
        if self.use_layer_scale:
            x = x + self.drop_path(
                self.layer_scale_1.unsqueeze(-1).unsqueeze(-1)
                * self.token_mixer(self.norm1(x)))
            x = x + self.drop_path(
                self.layer_scale_2.unsqueeze(-1).unsqueeze(-1)
                * self.mlp(self.norm2(x)))
        else:
            x = x + self.drop_path(self.token_mixer(self.norm1(x)))
            x = x + self.drop_path(self.mlp(self.norm2(x)))
        return x
```

算法4 PoolFormer模块，类似PyTorch的代码

```
import torch.nn as nn

class PoolFormerBlock(nn.Module):
    """
    Implementation of one PoolFormer block.
    --dim: embedding dim
    --pool_size: pooling size
    --mlp_ratio: mlp expansion ratio
    --act_layer: activation
    --norm_layer: normalization
    --drop: dropout rate
    --drop path: Stochastic Depth,
        refer to https://arxiv.org/abs/1603.09382
    --use_layer_scale, --layer_scale_init_value: LayerScale,
        refer to https://arxiv.org/abs/2103.17239
    """
    def __init__(self, dim, pool_size=3, mlp_ratio=4.,
                 act_layer=nn.GELU, norm_layer=ModifiedLayerNorm,
                 drop=0., drop_path=0.,
                 use_layer_scale=True, layer_scale_init_value=1e-5):

        super().init()

        self.norm1 = norm_layer(dim)
        self.token_mixer = Pooling(pool_size=pool_size)
        self.norm2 = norm_layer(dim)
        mlp_hidden_dim = int(dim * mlp_ratio)
        self.mlp = Mlp(in_features=dim, hidden_features=mlp_hidden_dim,
                       act_layer=act_layer, drop=drop)

        # The following two techniques are useful to train deep PoolFormers.
        self.drop_path = DropPath(drop_path) if drop_path > 0. \
            else nn.Identity()
        self.use_layer_scale = use_layer_scale
        if use_layer_scale:
            self.layer_scale_1 = nn.Parameter(
                layer_scale_init_value * torch.ones(dim), requires_grad=True)
            self.layer_scale_2 = nn.Parameter(
                layer_scale_init_value * torch.ones(dim), requires_grad=True)

    def forward(self, x):
        if self.use_layer_scale:
            x = x + self.drop_path(
                self.layer_scale_1.unsqueeze(-1).unsqueeze(-1)
                * self.token_mixer(self.norm1(x)))
            x = x + self.drop_path(
                self.layer_scale_2.unsqueeze(-1).unsqueeze(-1)
                * self.mlp(self.norm2(x)))
        else:
            x = x + self.drop_path(self.token_mixer(self.norm1(x)))
            x = x + self.drop_path(self.mlp(self.norm2(x)))
        return x
```